

DEEP LEARNING FOR FINANCE

MODULE 6 | LESSON 2

APPLICATION OF AUTOENCODERS: EXTRACTING FEATURES FROM STOCK RETURNS

Reading Time	50 minutes
Prior Knowledge	Neural Networks, machine learning
Keywords	Autoencoder, Principal Components, Factors

1. INTRODUCTION

In this notebook, we will describe how to build a basic autoencoder model with TensorFlow using stock returns data. The exercise here will illustrate the tools that we described in Lesson 1 and the typical outputs generated by autoencoders to understand the variation that drives security prices. In Lesson 4, we will extend the analysis allowing an autoencoder model to become a tool for portfolio choice.

Let's first load some libraries needed in this notebook altogether with the data. We are going to analyze 20 tech stocks and their evolution over the 2017-2022 period. Using autoencoders, we will extract the common encodings, or latent "factors", that determine the variation of stock returns and how each stock is exposed to those factors.

```
In [1]: import numpy as np
import pandas as pd
```

```
In [2]: # List of tickers (same order)
tickers = [
    "AAPL", "MSFT", "GOOG", "AMZN", "TSLA",
    "TCEHY", "NVDA", "TSM", "META", "BABA",
    "ASML", "AVGO", "ORCL", "CSCO", "ADBE",
    "CRM", "NFLX", "TXN", "QCOM", "SAP"
]

# Read CSV directly with Date as index
print("Loading data from CSV...")
data = pd.read_csv('adjusted_prices.csv', index_col='Date', parse_dates=True)

print(f"Data loaded: {data.shape}")
print(f"Complete range: {data.index[0].date()} to {data.index[-1].date()}")

# Calculate daily returns (pct_change) for all tickers
```

```
print("Calculating daily returns...")
returns_data = data[tickers].pct_change()

# Remove first row (NaN from pct_change)
df = returns_data.dropna()

df.head()
```

Loading data from CSV...

Data loaded: (1509, 20)

Complete range: 2019-01-02 to 2024-12-30

Calculating daily returns...

Out[2]:

	AAPL	MSFT	GOOG	AMZN	TSLA	TCEHY	NVDA	
Date								
2019-01-03 00:00:00-05:00	-0.099607	-0.036788	-0.028484	-0.025241	-0.031472	-0.028347	-0.060417	-0.040000
2019-01-04 00:00:00-05:00	0.042689	0.046509	0.053786	0.050064	0.057697	0.058088	0.064068	0.040000
2019-01-07 00:00:00-05:00	-0.002226	0.001275	-0.002167	0.034353	0.054361	0.006893	0.052940	0.040000
2019-01-08 00:00:00-05:00	0.019063	0.007251	0.007385	0.016612	0.001164	-0.003423	-0.024895	-0.040000
2019-01-09 00:00:00-05:00	0.016981	0.014300	-0.001505	0.001714	0.009483	0.038518	0.019667	0.040000

Use 80% of the data as a training sample (below we will use an additional 20% of the training data for validation purposes) and the remaining 20% belongs to the test part.

```
In [3]: print(f"Final DataFrame: {df.shape}")
print("Tickers included:", df.columns.tolist())

# Train/test split (80/20)
train_size = int(0.8 * df.shape[0])
Y_train = df.iloc[:train_size + 1].to_numpy()
Y_test = df.iloc[train_size + 1:].to_numpy()
```

Final DataFrame: (1508, 20)

Tickers included: ['AAPL', 'MSFT', 'GOOG', 'AMZN', 'TSLA', 'TCEHY', 'NVDA', 'TSM', 'META', 'BABA', 'ASML', 'AVGO', 'ORCL', 'CSCO', 'ADBE', 'CRM', 'NFLX', 'TXN', 'QCOM', 'SAP']

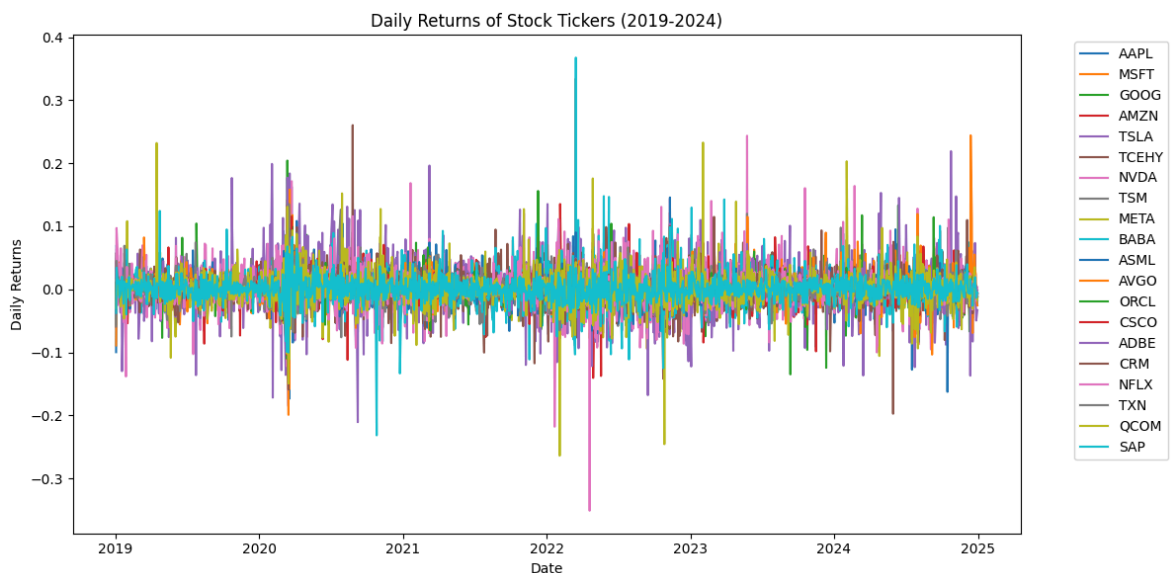
```
In [4]: print(f"\nBeginning of training sample: {df.index[0].date()}")
print(f"End of training sample: {df.index[train_size].date()}")
```

```
print(f"End of test sample: {df.index[-1].date()}")
print(f"Training samples: {Y_train.shape}")
print(f"Test samples: {Y_test.shape}")
```

Beginning of training sample: 2019-01-03
 End of training sample: 2023-10-18
 End of test sample: 2024-12-30
 Training samples: (1207, 20)
 Test samples: (301, 20)

In [5]: `import matplotlib.pyplot as plt`

```
plt.figure(figsize=(12, 6))
plt.plot(df.index, df)
plt.title(f'Daily Returns of Stock Tickers ({df.index[0].year}-{df.index[-1].year})')
plt.xlabel('Date')
plt.ylabel('Daily Returns')
plt.legend(df.columns, bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



2. APPLYING PRINCIPAL COMPONENT ANALYSIS (PCA)

Before building the autoencoder, we are going to perform principal component analysis (PCA) on the data. This analysis serves as a good benchmark when we later train autoencoders to learn the latent factors.

Remember that PCA consists on a linear transformation of the data to a low-dimensional space of factors that are orthogonal between each other. Below, we perform PCA on the data, extracting the first 3 principal components. The principal components are the columns of the matrix of eigenvectors from diagonalizing the variance-covariance matrix of the data. We have chosen the first 3 principal components to explain the stock return variation of 20 stocks, meaning that we obtain three 20x1 vectors of loadings. The loadings capture how much each single stock is "exposed" to the principal component.

We can also extract the proportion of the variation of the data explained by the first 3 principal components. See how the first principal component explains more than half of the variation in the data. The second and third components explain, respectively, roughly 10 percent and 5 percent of the variation.

The factors that drive stock returns and that are extracted from PCA represent latent variables that are sometimes hard to interpret or to give them a "name," particularly as we consider the second, third, and further components. However, when we analyze stock returns, the first principal component can be most of the time attributed to a "market factor" that generates positive comovement in returns across stocks. As we show, the loadings on this first factor are all negative across stocks (remember that the variance of $\$Z$ is the same as the variance of $\$-Z$). This confirms that the stock price of a single stock tends to move in the same direction as all the remaining stocks, and this explains more than 50% of the variation of returns.

```
In [6]: from sklearn.decomposition import PCA

pca = PCA(n_components=3)
pca.fit(Y_train)
print(pca.components_.T)
# Cumulative share of the variation of the data predicted the factors
print(np.cumsum(pca.explained_variance_ratio_))

[[-0.20332611 -0.08862559 -0.00286622]
 [-0.1996453  -0.13252331  0.01765695]
 [-0.19036866 -0.11370978  0.03904352]
 [-0.20425213 -0.06864972  0.00102689]
 [-0.3344536   0.47144555 -0.79461501]
 [-0.1901963   0.487918   0.37368002]
 [-0.3408636  -0.15417502 -0.02877177]
 [-0.20345294 -0.04641713  0.04437633]
 [-0.24227164 -0.11326835  0.10667999]
 [-0.21053143  0.57238145  0.44041048]
 [-0.26026589 -0.09684741  0.04523965]
 [-0.22736039 -0.12364523  0.00800747]
 [-0.13365969 -0.1512764   0.07292133]
 [-0.13219045 -0.14066919  0.05780975]
 [-0.23419577 -0.12478373  0.02581529]
 [-0.21978161 -0.08489683 -0.00770036]
 [-0.22835938  0.03082332  0.00750172]
 [-0.19340585 -0.13229862  0.05043233]
 [-0.24460667 -0.12143996  0.04582046]
 [-0.15779398 -0.0831969   0.05263593]]
[0.52145945 0.60819791 0.69199829]
```

Notice that the factors generated by PCA are indeed uncorrelated and that the loadings are orthonormal.

```
In [7]: # Factors (encodings) generated by PCA
pca_factors = np.dot(pca.components_, Y_train.T)
# Show that factors are uncorrelated
print(np.round(np.cov(pca_factors, rowvar=True), 5))
# Show that Loadings are orthonormal
print(np.round(np.dot(pca.components_, pca.components_.T), 5))
```

```
[[ 0.00672 -0.      -0.      ]
 [-0.      0.00112  0.      ]
 [-0.      0.      0.00108]]
[[ 1.  0. -0.]
 [ 0.  1. -0.]
 [-0. -0.  1.]]
```

An important remark is that the loadings and factors from PCA are not unique. Any linear, orthogonal transformation of the eigenvectors also maximizes the variance generated by the projections. Different software packages may generate different solutions. However, all the solutions should explain the same amount of variation in a given dataset.

Below we perform PCA extracting the first 6 principal components.

```
In [8]: pca = PCA(n_components=6)
pca.fit(Y_train)
print(pca.components_.T)
# Cumulative share of the variation of the data predicted the factors
print(np.cumsum(pca.explained_variance_ratio_))
```

```
[[-2.03326113e-01 -8.86305513e-02 -2.86892556e-03 -2.18050311e-03
  1.39588981e-01 1.28823724e-02]
 [-1.99645303e-01 -1.32516970e-01 1.76537624e-02 -4.81170301e-02
  1.71106411e-01 5.93128190e-02]
 [-1.90368657e-01 -1.13710192e-01 3.90304093e-02 -1.16007207e-01
  1.85887877e-01 -1.35038202e-01]
 [-2.04252134e-01 -6.86576446e-02 1.03641457e-03 -2.39601579e-01
  4.00280187e-02 -1.01450360e-01]
 [-3.34453600e-01 4.71445859e-01 -7.94615233e-01 4.42999102e-02
  1.25302888e-01 4.66791209e-04]
 [-1.90196303e-01 4.87912683e-01 3.73670019e-01 1.13861766e-01
  1.42086030e-02 5.28128873e-02]
 [-3.40863598e-01 -1.54182844e-01 -2.87800844e-02 1.77631264e-01
  -3.02636192e-01 -2.48178199e-01]
 [-2.03452942e-01 -4.64344566e-02 4.43868884e-02 2.80096037e-01
  -2.01393272e-01 -7.76834049e-02]
 [-2.42271644e-01 -1.13266798e-01 1.06681564e-01 -3.87717440e-01
  2.42011811e-01 -6.54256789e-01]
 [-2.10531430e-01 5.72387453e-01 4.40417989e-01 -6.95735330e-03
  9.04838690e-02 -1.43220865e-02]
 [-2.60265891e-01 -9.68224394e-02 4.52474096e-02 2.35928622e-01
  -1.61806236e-01 -2.68969329e-02]
 [-2.27360388e-01 -1.23630789e-01 8.01110306e-03 2.37829369e-01
  -5.97119217e-02 8.00429060e-02]
 [-1.33659691e-01 -1.51271071e-01 7.29213556e-02 1.08568379e-02
  2.54283356e-01 3.95803673e-01]
 [-1.32190446e-01 -1.40679185e-01 5.78071144e-02 7.91530861e-02
  2.17840845e-01 2.46379970e-01]
 [-2.34195770e-01 -1.24760700e-01 2.58375904e-02 -1.10667433e-01
  1.49822631e-01 1.56191581e-01]
 [-2.19781612e-01 -8.49051109e-02 -7.70505031e-03 -2.04613672e-01
  2.21465526e-01 2.42335076e-01]
 [-2.28359384e-01 3.08221802e-02 7.49818855e-03 -6.11911575e-01
  -6.34343821e-01 3.21201283e-01]
 [-1.93405853e-01 -1.32295528e-01 5.04350341e-02 1.88358479e-01
  -4.28439101e-02 6.10902382e-02]
 [-2.44606667e-01 -1.21448248e-01 4.58119833e-02 2.59948448e-01
  -2.13862509e-01 -8.99871562e-02]
 [-1.57793978e-01 -8.32232783e-02 5.26238628e-02 3.60058548e-02
  1.92232203e-01 2.15636056e-01]]
[0.52145945 0.60819791 0.69199829 0.7478485 0.78188589 0.81024463]
```

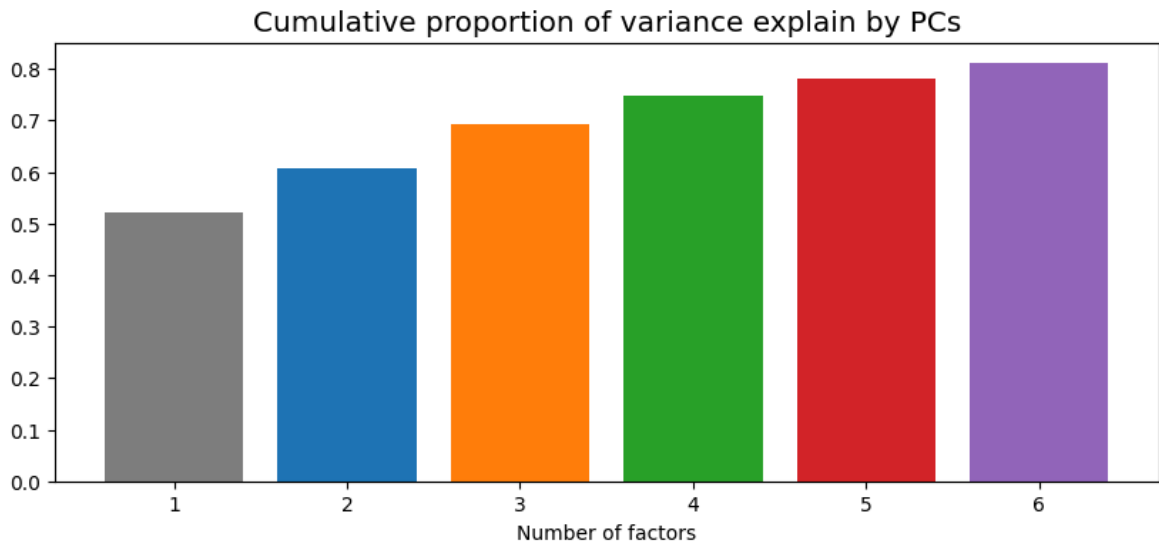
You can verify that the solution with 3 principal components is nested by the solution when we extract additional principal components. That is, the loadings on the first three components and the variation explained by them is the same as we obtained above.

```
In [9]: # plotting the coefficient score
fig, ax = plt.subplots(figsize=(10, 4))
color = [
    "tab:gray",
    "tab:blue",
    "tab:orange",
    "tab:green",
    "tab:red",
    "tab:purple",
    "tab:brown",
    "tab:pink",
    "tab:gray",
    "tab:olive",
```

```

    "tab:cyan",
    "tab:orange",
    "tab:green",
    "tab:blue",
    "tab:olive",
]
ax.bar(list(range(1, 7)), np.cumsum(pca.explained_variance_ratio_), color=color)
plt.style.use("ggplot")
plt.title("Cumulative proportion of variance explain by PCs")
plt.xlabel("Number of factors")
plt.show()

```



3. BUILDING A LINEAR AUTOENCODER WITH TENSORFLOW

Below we build a single-layer linear autoencoder with TensorFlow via [subclassing](#). The autoencoder has two Dense (fully connected) layers: an encoder, which compresses the data into a latent vector with a dimension set by "bottleneck_dim," and a decoder, which reconstructs the original input from the latent space.

In [10]: `import tensorflow as tf`

```

2025-05-26 10:46:01.846294: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-05-26 10:46:01.890793: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI AVX512_BF16 AVX512_FP16 AVX_VNNI AMX_TILE AMX_INT8 AMX_BF16 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
2025-05-26 10:46:03.404539: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.

```

In [11]: `class autoencoder(tf.keras.Model):`
`def __init__(self, output_dim, bottleneck_dim=1):`
`super(autoencoder, self).__init__()`

```

self.output_dim = output_dim
self.bottleneck_dim = bottleneck_dim
initializer = tf.keras.initializers.GlorotUniform()
# Build the encoder layer
self.encoder = tf.keras.Sequential(
    [
        tf.keras.layers.Dense(
            bottleneck_dim,
            kernel_initializer=initializer,
            bias_initializer=tf.keras.initializers.Zeros(),
        )
    ]
)
# Build the decoder layer
self.decoder = tf.keras.Sequential(
    [
        tf.keras.layers.Dense(
            output_dim,
            kernel_initializer=initializer,
            bias_initializer=tf.keras.initializers.Zeros(),
        )
    ]
)

def call(self, x):
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    return decoded

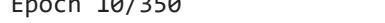
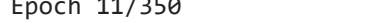
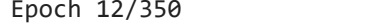
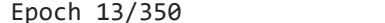
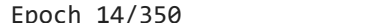
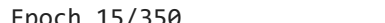
```

Below we compile and train the model using our training data. We choose a "bottleneck" with 3 latent factors. We train the model for at most 1,000 epochs, setting an early stopping criterion using a validation sample of 20% of all the training observations. The early stopping criterion may not bind, depending on the particular seed used in the training algorithm.

```

In [12]: # Choose the number of latent factors
nlatent = 3
# Choose the proportion of training data used for validation
val_split = 0.2
# Determine early stopping criterion
early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=50)
# Choose optimizer
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
# Build and compile the model
model = autoencoder(Y_train.shape[1], nlatent)
model.compile(optimizer=adam, loss="mse")
# Train the model
model.fit(
    x=Y_train,
    y=Y_train,
    batch_size=32,
    epochs=350,
    validation_split=val_split,
    callbacks=[early_stop],
    verbose=1,
)

```

```
Epoch 1/350
31/31  1s 7ms/step - loss: 9.2919e-04 - val_loss: 7.5377e-04
Epoch 2/350
31/31  0s 4ms/step - loss: 9.0401e-04 - val_loss: 7.3931e-04
Epoch 3/350
31/31  0s 4ms/step - loss: 8.4248e-04 - val_loss: 7.2670e-04
Epoch 4/350
31/31  0s 4ms/step - loss: 8.6337e-04 - val_loss: 7.1399e-04
Epoch 5/350
31/31  0s 4ms/step - loss: 9.2274e-04 - val_loss: 7.0303e-04
Epoch 6/350
31/31  0s 4ms/step - loss: 8.8731e-04 - val_loss: 6.9276e-04
Epoch 7/350
31/31  0s 4ms/step - loss: 7.6780e-04 - val_loss: 6.8293e-04
Epoch 8/350
31/31  0s 3ms/step - loss: 8.1681e-04 - val_loss: 6.7424e-04
Epoch 9/350
31/31  0s 3ms/step - loss: 8.1545e-04 - val_loss: 6.6614e-04
Epoch 10/350
31/31  0s 3ms/step - loss: 8.1747e-04 - val_loss: 6.5837e-04
Epoch 11/350
31/31  0s 4ms/step - loss: 8.1108e-04 - val_loss: 6.5109e-04
Epoch 12/350
31/31  0s 4ms/step - loss: 8.1877e-04 - val_loss: 6.4440e-04
Epoch 13/350
31/31  0s 4ms/step - loss: 7.9767e-04 - val_loss: 6.3781e-04
Epoch 14/350
31/31  0s 4ms/step - loss: 7.8676e-04 - val_loss: 6.3212e-04
Epoch 15/350
31/31  0s 4ms/step - loss: 7.2650e-04 - val_loss: 6.2640e-04
Epoch 16/350
31/31  0s 4ms/step - loss: 8.4396e-04 - val_loss: 6.2102e-04
Epoch 17/350
31/31  0s 4ms/step - loss: 7.5019e-04 - val_loss: 6.1572e-04
Epoch 18/350
31/31  0s 3ms/step - loss: 7.1734e-04 - val_loss: 6.1096e-04
Epoch 19/350
31/31  0s 3ms/step - loss: 6.8568e-04 - val_loss: 6.0638e-04
Epoch 20/350
31/31  0s 4ms/step - loss: 7.2140e-04 - val_loss: 6.0150e-04
Epoch 21/350
31/31  0s 4ms/step - loss: 7.4381e-04 - val_loss: 5.9690e-04
Epoch 22/350
31/31  0s 4ms/step - loss: 6.6287e-04 - val_loss: 5.9267e-04
Epoch 23/350
31/31  0s 4ms/step - loss: 6.9261e-04 - val_loss: 5.8892e-04
Epoch 24/350
31/31  0s 4ms/step - loss: 6.9214e-04 - val_loss: 5.8469e-04
Epoch 25/350
31/31  0s 4ms/step - loss: 6.7579e-04 - val_loss: 5.8076e-04
Epoch 26/350
31/31  0s 4ms/step - loss: 6.8817e-04 - val_loss: 5.7696e-04
Epoch 27/350
31/31  0s 3ms/step - loss: 6.8490e-04 - val_loss: 5.7286e-04
Epoch 28/350
31/31  0s 3ms/step - loss: 6.1713e-04 - val_loss: 5.6921e-04
Epoch 29/350
31/31  0s 4ms/step - loss: 6.7855e-04 - val_loss: 5.6550e-04
Epoch 30/350
31/31  0s 4ms/step - loss: 6.6751e-04 - val_loss: 5.6142e-04
```

```
Epoch 31/350
31/31  0s 4ms/step - loss: 6.4237e-04 - val_loss: 5.5770e-04
Epoch 32/350
31/31  0s 4ms/step - loss: 6.4744e-04 - val_loss: 5.5383e-04
Epoch 33/350
31/31  0s 4ms/step - loss: 6.6856e-04 - val_loss: 5.5003e-04
Epoch 34/350
31/31  0s 4ms/step - loss: 6.3021e-04 - val_loss: 5.4650e-04
Epoch 35/350
31/31  0s 4ms/step - loss: 6.3122e-04 - val_loss: 5.4242e-04
Epoch 36/350
31/31  0s 4ms/step - loss: 6.4134e-04 - val_loss: 5.3859e-04
Epoch 37/350
31/31  0s 4ms/step - loss: 6.0461e-04 - val_loss: 5.3535e-04
Epoch 38/350
31/31  0s 4ms/step - loss: 6.1082e-04 - val_loss: 5.3173e-04
Epoch 39/350
31/31  0s 4ms/step - loss: 6.8435e-04 - val_loss: 5.2781e-04
Epoch 40/350
31/31  0s 4ms/step - loss: 5.9851e-04 - val_loss: 5.2407e-04
Epoch 41/350
31/31  0s 4ms/step - loss: 5.9480e-04 - val_loss: 5.2038e-04
Epoch 42/350
31/31  0s 4ms/step - loss: 6.1261e-04 - val_loss: 5.1671e-04
Epoch 43/350
31/31  0s 4ms/step - loss: 6.3842e-04 - val_loss: 5.1291e-04
Epoch 44/350
31/31  0s 4ms/step - loss: 5.7130e-04 - val_loss: 5.0945e-04
Epoch 45/350
31/31  0s 4ms/step - loss: 5.7881e-04 - val_loss: 5.0573e-04
Epoch 46/350
31/31  0s 3ms/step - loss: 6.1606e-04 - val_loss: 5.0204e-04
Epoch 47/350
31/31  0s 4ms/step - loss: 5.4292e-04 - val_loss: 4.9840e-04
Epoch 48/350
31/31  0s 4ms/step - loss: 5.8006e-04 - val_loss: 4.9462e-04
Epoch 49/350
31/31  0s 3ms/step - loss: 5.7763e-04 - val_loss: 4.9101e-04
Epoch 50/350
31/31  0s 3ms/step - loss: 5.4772e-04 - val_loss: 4.8748e-04
Epoch 51/350
31/31  0s 3ms/step - loss: 5.7466e-04 - val_loss: 4.8358e-04
Epoch 52/350
31/31  0s 4ms/step - loss: 5.4748e-04 - val_loss: 4.8021e-04
Epoch 53/350
31/31  0s 4ms/step - loss: 5.4614e-04 - val_loss: 4.7637e-04
Epoch 54/350
31/31  0s 4ms/step - loss: 5.3049e-04 - val_loss: 4.7280e-04
Epoch 55/350
31/31  0s 4ms/step - loss: 5.3070e-04 - val_loss: 4.6932e-04
Epoch 56/350
31/31  0s 4ms/step - loss: 5.0853e-04 - val_loss: 4.6543e-04
Epoch 57/350
31/31  0s 4ms/step - loss: 5.1897e-04 - val_loss: 4.6169e-04
Epoch 58/350
31/31  0s 4ms/step - loss: 5.4831e-04 - val_loss: 4.5808e-04
Epoch 59/350
31/31  0s 4ms/step - loss: 4.6257e-04 - val_loss: 4.5483e-04
Epoch 60/350
31/31  0s 4ms/step - loss: 5.2082e-04 - val_loss: 4.5117e-04
```

```
Epoch 61/350
31/31  0s 4ms/step - loss: 4.8605e-04 - val_loss: 4.4763e-04
Epoch 62/350
31/31  0s 4ms/step - loss: 5.0857e-04 - val_loss: 4.4419e-04
Epoch 63/350
31/31  0s 4ms/step - loss: 4.9395e-04 - val_loss: 4.4064e-04
Epoch 64/350
31/31  0s 5ms/step - loss: 5.0147e-04 - val_loss: 4.3713e-04
Epoch 65/350
31/31  0s 4ms/step - loss: 4.7816e-04 - val_loss: 4.3351e-04
Epoch 66/350
31/31  0s 4ms/step - loss: 5.0089e-04 - val_loss: 4.3027e-04
Epoch 67/350
31/31  0s 4ms/step - loss: 4.9325e-04 - val_loss: 4.2655e-04
Epoch 68/350
31/31  0s 4ms/step - loss: 4.9347e-04 - val_loss: 4.2344e-04
Epoch 69/350
31/31  0s 4ms/step - loss: 4.6400e-04 - val_loss: 4.1986e-04
Epoch 70/350
31/31  0s 4ms/step - loss: 4.6083e-04 - val_loss: 4.1635e-04
Epoch 71/350
31/31  0s 4ms/step - loss: 4.6817e-04 - val_loss: 4.1292e-04
Epoch 72/350
31/31  0s 4ms/step - loss: 4.4257e-04 - val_loss: 4.0943e-04
Epoch 73/350
31/31  0s 4ms/step - loss: 4.6016e-04 - val_loss: 4.0576e-04
Epoch 74/350
31/31  0s 4ms/step - loss: 4.4633e-04 - val_loss: 4.0281e-04
Epoch 75/350
31/31  0s 4ms/step - loss: 4.8629e-04 - val_loss: 3.9924e-04
Epoch 76/350
31/31  0s 4ms/step - loss: 4.8398e-04 - val_loss: 3.9635e-04
Epoch 77/350
31/31  0s 4ms/step - loss: 3.9727e-04 - val_loss: 3.9289e-04
Epoch 78/350
31/31  0s 3ms/step - loss: 4.1785e-04 - val_loss: 3.8952e-04
Epoch 79/350
31/31  0s 4ms/step - loss: 4.4408e-04 - val_loss: 3.8617e-04
Epoch 80/350
31/31  0s 4ms/step - loss: 4.3119e-04 - val_loss: 3.8283e-04
Epoch 81/350
31/31  0s 4ms/step - loss: 4.2323e-04 - val_loss: 3.7961e-04
Epoch 82/350
31/31  0s 4ms/step - loss: 4.1451e-04 - val_loss: 3.7675e-04
Epoch 83/350
31/31  0s 4ms/step - loss: 4.1517e-04 - val_loss: 3.7340e-04
Epoch 84/350
31/31  0s 4ms/step - loss: 4.0943e-04 - val_loss: 3.7014e-04
Epoch 85/350
31/31  0s 4ms/step - loss: 4.1974e-04 - val_loss: 3.6712e-04
Epoch 86/350
31/31  0s 4ms/step - loss: 4.2334e-04 - val_loss: 3.6435e-04
Epoch 87/350
31/31  0s 4ms/step - loss: 4.1801e-04 - val_loss: 3.6121e-04
Epoch 88/350
31/31  0s 4ms/step - loss: 3.9369e-04 - val_loss: 3.5826e-04
Epoch 89/350
31/31  0s 4ms/step - loss: 4.1828e-04 - val_loss: 3.5541e-04
Epoch 90/350
31/31  0s 4ms/step - loss: 4.2806e-04 - val_loss: 3.5233e-04
```

```
Epoch 91/350
31/31  0s 4ms/step - loss: 4.1618e-04 - val_loss: 3.4964e-04
Epoch 92/350
31/31  0s 4ms/step - loss: 3.9535e-04 - val_loss: 3.4657e-04
Epoch 93/350
31/31  0s 4ms/step - loss: 3.7432e-04 - val_loss: 3.4396e-04
Epoch 94/350
31/31  0s 4ms/step - loss: 3.7788e-04 - val_loss: 3.4136e-04
Epoch 95/350
31/31  0s 4ms/step - loss: 3.4525e-04 - val_loss: 3.3849e-04
Epoch 96/350
31/31  0s 4ms/step - loss: 3.8217e-04 - val_loss: 3.3589e-04
Epoch 97/350
31/31  0s 4ms/step - loss: 3.8639e-04 - val_loss: 3.3339e-04
Epoch 98/350
31/31  0s 4ms/step - loss: 3.7903e-04 - val_loss: 3.3091e-04
Epoch 99/350
31/31  0s 4ms/step - loss: 3.6542e-04 - val_loss: 3.2864e-04
Epoch 100/350
31/31  0s 4ms/step - loss: 3.3699e-04 - val_loss: 3.2649e-04
Epoch 101/350
31/31  0s 4ms/step - loss: 3.5413e-04 - val_loss: 3.2395e-04
Epoch 102/350
31/31  0s 4ms/step - loss: 3.5483e-04 - val_loss: 3.2177e-04
Epoch 103/350
31/31  0s 5ms/step - loss: 3.4037e-04 - val_loss: 3.1961e-04
Epoch 104/350
31/31  0s 4ms/step - loss: 3.2986e-04 - val_loss: 3.1754e-04
Epoch 105/350
31/31  0s 4ms/step - loss: 3.3308e-04 - val_loss: 3.1543e-04
Epoch 106/350
31/31  0s 4ms/step - loss: 3.4204e-04 - val_loss: 3.1346e-04
Epoch 107/350
31/31  0s 4ms/step - loss: 3.4981e-04 - val_loss: 3.1108e-04
Epoch 108/350
31/31  0s 4ms/step - loss: 3.3718e-04 - val_loss: 3.0934e-04
Epoch 109/350
31/31  0s 4ms/step - loss: 3.4511e-04 - val_loss: 3.0761e-04
Epoch 110/350
31/31  0s 4ms/step - loss: 3.1178e-04 - val_loss: 3.0590e-04
Epoch 111/350
31/31  0s 4ms/step - loss: 3.1879e-04 - val_loss: 3.0442e-04
Epoch 112/350
31/31  0s 3ms/step - loss: 3.2512e-04 - val_loss: 3.0249e-04
Epoch 113/350
31/31  0s 3ms/step - loss: 3.2146e-04 - val_loss: 3.0099e-04
Epoch 114/350
31/31  0s 4ms/step - loss: 3.2598e-04 - val_loss: 2.9937e-04
Epoch 115/350
31/31  0s 4ms/step - loss: 3.2396e-04 - val_loss: 2.9784e-04
Epoch 116/350
31/31  0s 4ms/step - loss: 3.1402e-04 - val_loss: 2.9641e-04
Epoch 117/350
31/31  0s 3ms/step - loss: 3.0830e-04 - val_loss: 2.9478e-04
Epoch 118/350
31/31  0s 4ms/step - loss: 2.9192e-04 - val_loss: 2.9323e-04
Epoch 119/350
31/31  0s 4ms/step - loss: 3.0044e-04 - val_loss: 2.9223e-04
Epoch 120/350
31/31  0s 4ms/step - loss: 3.2041e-04 - val_loss: 2.9080e-04
```

```
Epoch 121/350
31/31  0s 4ms/step - loss: 2.9978e-04 - val_loss: 2.8943e-04
Epoch 122/350
31/31  0s 4ms/step - loss: 3.0148e-04 - val_loss: 2.8807e-04
Epoch 123/350
31/31  0s 4ms/step - loss: 3.0802e-04 - val_loss: 2.8688e-04
Epoch 124/350
31/31  0s 4ms/step - loss: 3.3759e-04 - val_loss: 2.8565e-04
Epoch 125/350
31/31  0s 4ms/step - loss: 3.0873e-04 - val_loss: 2.8480e-04
Epoch 126/350
31/31  0s 4ms/step - loss: 3.0583e-04 - val_loss: 2.8332e-04
Epoch 127/350
31/31  0s 4ms/step - loss: 2.9842e-04 - val_loss: 2.8229e-04
Epoch 128/350
31/31  0s 4ms/step - loss: 3.1516e-04 - val_loss: 2.8133e-04
Epoch 129/350
31/31  0s 4ms/step - loss: 2.9283e-04 - val_loss: 2.8027e-04
Epoch 130/350
31/31  0s 3ms/step - loss: 2.9061e-04 - val_loss: 2.7938e-04
Epoch 131/350
31/31  0s 4ms/step - loss: 3.0848e-04 - val_loss: 2.7841e-04
Epoch 132/350
31/31  0s 4ms/step - loss: 2.9066e-04 - val_loss: 2.7748e-04
Epoch 133/350
31/31  0s 3ms/step - loss: 2.8932e-04 - val_loss: 2.7647e-04
Epoch 134/350
31/31  0s 4ms/step - loss: 2.9497e-04 - val_loss: 2.7555e-04
Epoch 135/350
31/31  0s 4ms/step - loss: 2.9395e-04 - val_loss: 2.7474e-04
Epoch 136/350
31/31  0s 4ms/step - loss: 3.1819e-04 - val_loss: 2.7409e-04
Epoch 137/350
31/31  0s 4ms/step - loss: 2.8042e-04 - val_loss: 2.7303e-04
Epoch 138/350
31/31  0s 5ms/step - loss: 2.9634e-04 - val_loss: 2.7220e-04
Epoch 139/350
31/31  0s 3ms/step - loss: 2.7763e-04 - val_loss: 2.7167e-04
Epoch 140/350
31/31  0s 4ms/step - loss: 2.9456e-04 - val_loss: 2.7072e-04
Epoch 141/350
31/31  0s 4ms/step - loss: 3.1669e-04 - val_loss: 2.6984e-04
Epoch 142/350
31/31  0s 4ms/step - loss: 3.0520e-04 - val_loss: 2.6929e-04
Epoch 143/350
31/31  0s 4ms/step - loss: 2.7530e-04 - val_loss: 2.6838e-04
Epoch 144/350
31/31  0s 4ms/step - loss: 2.6872e-04 - val_loss: 2.6786e-04
Epoch 145/350
31/31  0s 3ms/step - loss: 2.6615e-04 - val_loss: 2.6707e-04
Epoch 146/350
31/31  0s 4ms/step - loss: 2.7653e-04 - val_loss: 2.6634e-04
Epoch 147/350
31/31  0s 3ms/step - loss: 2.7918e-04 - val_loss: 2.6569e-04
Epoch 148/350
31/31  0s 4ms/step - loss: 2.8134e-04 - val_loss: 2.6496e-04
Epoch 149/350
31/31  0s 4ms/step - loss: 2.7893e-04 - val_loss: 2.6437e-04
Epoch 150/350
31/31  0s 3ms/step - loss: 2.9071e-04 - val_loss: 2.6380e-04
```

```
Epoch 151/350
31/31  0s 4ms/step - loss: 2.7412e-04 - val_loss: 2.6298e-04
Epoch 152/350
31/31  0s 4ms/step - loss: 2.8317e-04 - val_loss: 2.6229e-04
Epoch 153/350
31/31  0s 4ms/step - loss: 2.7402e-04 - val_loss: 2.6170e-04
Epoch 154/350
31/31  0s 4ms/step - loss: 2.9113e-04 - val_loss: 2.6110e-04
Epoch 155/350
31/31  0s 4ms/step - loss: 2.8000e-04 - val_loss: 2.6029e-04
Epoch 156/350
31/31  0s 4ms/step - loss: 2.7153e-04 - val_loss: 2.5981e-04
Epoch 157/350
31/31  0s 4ms/step - loss: 2.6143e-04 - val_loss: 2.5924e-04
Epoch 158/350
31/31  0s 4ms/step - loss: 2.7795e-04 - val_loss: 2.5843e-04
Epoch 159/350
31/31  0s 3ms/step - loss: 2.9790e-04 - val_loss: 2.5786e-04
Epoch 160/350
31/31  0s 4ms/step - loss: 2.7053e-04 - val_loss: 2.5743e-04
Epoch 161/350
31/31  0s 3ms/step - loss: 2.7532e-04 - val_loss: 2.5680e-04
Epoch 162/350
31/31  0s 3ms/step - loss: 2.6738e-04 - val_loss: 2.5611e-04
Epoch 163/350
31/31  0s 4ms/step - loss: 2.9914e-04 - val_loss: 2.5548e-04
Epoch 164/350
31/31  0s 4ms/step - loss: 2.7534e-04 - val_loss: 2.5510e-04
Epoch 165/350
31/31  0s 4ms/step - loss: 2.6316e-04 - val_loss: 2.5431e-04
Epoch 166/350
31/31  0s 4ms/step - loss: 2.7122e-04 - val_loss: 2.5388e-04
Epoch 167/350
31/31  0s 3ms/step - loss: 2.6221e-04 - val_loss: 2.5332e-04
Epoch 168/350
31/31  0s 4ms/step - loss: 2.5142e-04 - val_loss: 2.5279e-04
Epoch 169/350
31/31  0s 4ms/step - loss: 2.6817e-04 - val_loss: 2.5208e-04
Epoch 170/350
31/31  0s 4ms/step - loss: 2.7834e-04 - val_loss: 2.5152e-04
Epoch 171/350
31/31  0s 4ms/step - loss: 2.5123e-04 - val_loss: 2.5122e-04
Epoch 172/350
31/31  0s 3ms/step - loss: 2.6014e-04 - val_loss: 2.5049e-04
Epoch 173/350
31/31  0s 4ms/step - loss: 2.5680e-04 - val_loss: 2.4998e-04
Epoch 174/350
31/31  0s 4ms/step - loss: 2.8767e-04 - val_loss: 2.4915e-04
Epoch 175/350
31/31  0s 4ms/step - loss: 2.5986e-04 - val_loss: 2.4899e-04
Epoch 176/350
31/31  0s 4ms/step - loss: 2.6431e-04 - val_loss: 2.4853e-04
Epoch 177/350
31/31  0s 3ms/step - loss: 2.6315e-04 - val_loss: 2.4809e-04
Epoch 178/350
31/31  0s 4ms/step - loss: 2.6235e-04 - val_loss: 2.4774e-04
Epoch 179/350
31/31  0s 4ms/step - loss: 2.7654e-04 - val_loss: 2.4686e-04
Epoch 180/350
31/31  0s 4ms/step - loss: 2.3677e-04 - val_loss: 2.4625e-04
```

```
Epoch 181/350
31/31  0s 4ms/step - loss: 2.4118e-04 - val_loss: 2.4573e-04
Epoch 182/350
31/31  0s 4ms/step - loss: 2.6377e-04 - val_loss: 2.4525e-04
Epoch 183/350
31/31  0s 4ms/step - loss: 2.6628e-04 - val_loss: 2.4480e-04
Epoch 184/350
31/31  0s 4ms/step - loss: 2.7813e-04 - val_loss: 2.4446e-04
Epoch 185/350
31/31  0s 4ms/step - loss: 2.5880e-04 - val_loss: 2.4380e-04
Epoch 186/350
31/31  0s 3ms/step - loss: 2.5003e-04 - val_loss: 2.4333e-04
Epoch 187/350
31/31  0s 3ms/step - loss: 2.6732e-04 - val_loss: 2.4270e-04
Epoch 188/350
31/31  0s 4ms/step - loss: 2.5005e-04 - val_loss: 2.4226e-04
Epoch 189/350
31/31  0s 3ms/step - loss: 2.6316e-04 - val_loss: 2.4163e-04
Epoch 190/350
31/31  0s 4ms/step - loss: 2.5228e-04 - val_loss: 2.4113e-04
Epoch 191/350
31/31  0s 4ms/step - loss: 2.4807e-04 - val_loss: 2.4077e-04
Epoch 192/350
31/31  0s 4ms/step - loss: 2.5585e-04 - val_loss: 2.4019e-04
Epoch 193/350
31/31  0s 4ms/step - loss: 2.6429e-04 - val_loss: 2.3985e-04
Epoch 194/350
31/31  0s 4ms/step - loss: 2.3686e-04 - val_loss: 2.3929e-04
Epoch 195/350
31/31  0s 4ms/step - loss: 2.5530e-04 - val_loss: 2.3867e-04
Epoch 196/350
31/31  0s 4ms/step - loss: 2.5364e-04 - val_loss: 2.3842e-04
Epoch 197/350
31/31  0s 4ms/step - loss: 2.4861e-04 - val_loss: 2.3746e-04
Epoch 198/350
31/31  0s 4ms/step - loss: 2.4805e-04 - val_loss: 2.3743e-04
Epoch 199/350
31/31  0s 4ms/step - loss: 2.5497e-04 - val_loss: 2.3679e-04
Epoch 200/350
31/31  0s 4ms/step - loss: 2.4203e-04 - val_loss: 2.3681e-04
Epoch 201/350
31/31  0s 3ms/step - loss: 2.5512e-04 - val_loss: 2.3597e-04
Epoch 202/350
31/31  0s 5ms/step - loss: 2.4466e-04 - val_loss: 2.3555e-04
Epoch 203/350
31/31  0s 3ms/step - loss: 2.6862e-04 - val_loss: 2.3504e-04
Epoch 204/350
31/31  0s 3ms/step - loss: 2.4463e-04 - val_loss: 2.3458e-04
Epoch 205/350
31/31  0s 3ms/step - loss: 2.4368e-04 - val_loss: 2.3417e-04
Epoch 206/350
31/31  0s 4ms/step - loss: 2.5015e-04 - val_loss: 2.3349e-04
Epoch 207/350
31/31  0s 4ms/step - loss: 2.5718e-04 - val_loss: 2.3324e-04
Epoch 208/350
31/31  0s 4ms/step - loss: 2.6617e-04 - val_loss: 2.3274e-04
Epoch 209/350
31/31  0s 4ms/step - loss: 2.3474e-04 - val_loss: 2.3237e-04
Epoch 210/350
31/31  0s 4ms/step - loss: 2.2944e-04 - val_loss: 2.3184e-04
```

```
Epoch 211/350
31/31  0s 4ms/step - loss: 2.4875e-04 - val_loss: 2.3159e-04
Epoch 212/350
31/31  0s 4ms/step - loss: 2.2187e-04 - val_loss: 2.3111e-04
Epoch 213/350
31/31  0s 4ms/step - loss: 2.5539e-04 - val_loss: 2.3026e-04
Epoch 214/350
31/31  0s 3ms/step - loss: 2.3744e-04 - val_loss: 2.3036e-04
Epoch 215/350
31/31  0s 4ms/step - loss: 2.3950e-04 - val_loss: 2.2968e-04
Epoch 216/350
31/31  0s 4ms/step - loss: 2.2768e-04 - val_loss: 2.2915e-04
Epoch 217/350
31/31  0s 4ms/step - loss: 2.4342e-04 - val_loss: 2.2868e-04
Epoch 218/350
31/31  0s 4ms/step - loss: 2.5912e-04 - val_loss: 2.2843e-04
Epoch 219/350
31/31  0s 4ms/step - loss: 2.3217e-04 - val_loss: 2.2813e-04
Epoch 220/350
31/31  0s 4ms/step - loss: 2.3110e-04 - val_loss: 2.2767e-04
Epoch 221/350
31/31  0s 4ms/step - loss: 2.4011e-04 - val_loss: 2.2723e-04
Epoch 222/350
31/31  0s 4ms/step - loss: 2.5601e-04 - val_loss: 2.2669e-04
Epoch 223/350
31/31  0s 5ms/step - loss: 2.2960e-04 - val_loss: 2.2627e-04
Epoch 224/350
31/31  0s 5ms/step - loss: 2.2889e-04 - val_loss: 2.2597e-04
Epoch 225/350
31/31  0s 5ms/step - loss: 2.3126e-04 - val_loss: 2.2532e-04
Epoch 226/350
31/31  0s 4ms/step - loss: 2.4700e-04 - val_loss: 2.2498e-04
Epoch 227/350
31/31  0s 3ms/step - loss: 2.4021e-04 - val_loss: 2.2449e-04
Epoch 228/350
31/31  0s 3ms/step - loss: 2.4193e-04 - val_loss: 2.2434e-04
Epoch 229/350
31/31  0s 3ms/step - loss: 2.3008e-04 - val_loss: 2.2408e-04
Epoch 230/350
31/31  0s 5ms/step - loss: 2.2903e-04 - val_loss: 2.2374e-04
Epoch 231/350
31/31  0s 3ms/step - loss: 2.1749e-04 - val_loss: 2.2329e-04
Epoch 232/350
31/31  0s 4ms/step - loss: 2.1405e-04 - val_loss: 2.2272e-04
Epoch 233/350
31/31  0s 4ms/step - loss: 2.2114e-04 - val_loss: 2.2250e-04
Epoch 234/350
31/31  0s 4ms/step - loss: 2.1744e-04 - val_loss: 2.2224e-04
Epoch 235/350
31/31  0s 4ms/step - loss: 2.1687e-04 - val_loss: 2.2182e-04
Epoch 236/350
31/31  0s 4ms/step - loss: 2.3701e-04 - val_loss: 2.2141e-04
Epoch 237/350
31/31  0s 3ms/step - loss: 2.4453e-04 - val_loss: 2.2126e-04
Epoch 238/350
31/31  0s 4ms/step - loss: 2.3995e-04 - val_loss: 2.2090e-04
Epoch 239/350
31/31  0s 4ms/step - loss: 2.2271e-04 - val_loss: 2.2052e-04
Epoch 240/350
31/31  0s 4ms/step - loss: 2.2968e-04 - val_loss: 2.2010e-04
```

```
Epoch 241/350
31/31  0s 4ms/step - loss: 2.3475e-04 - val_loss: 2.1975e-04
Epoch 242/350
31/31  0s 4ms/step - loss: 2.2368e-04 - val_loss: 2.1946e-04
Epoch 243/350
31/31  0s 4ms/step - loss: 2.2646e-04 - val_loss: 2.1929e-04
Epoch 244/350
31/31  0s 4ms/step - loss: 2.3054e-04 - val_loss: 2.1867e-04
Epoch 245/350
31/31  0s 4ms/step - loss: 2.1360e-04 - val_loss: 2.1846e-04
Epoch 246/350
31/31  0s 4ms/step - loss: 2.2103e-04 - val_loss: 2.1804e-04
Epoch 247/350
31/31  0s 4ms/step - loss: 2.3626e-04 - val_loss: 2.1771e-04
Epoch 248/350
31/31  0s 4ms/step - loss: 2.2420e-04 - val_loss: 2.1752e-04
Epoch 249/350
31/31  0s 4ms/step - loss: 2.2959e-04 - val_loss: 2.1734e-04
Epoch 250/350
31/31  0s 4ms/step - loss: 2.2242e-04 - val_loss: 2.1683e-04
Epoch 251/350
31/31  0s 4ms/step - loss: 2.2956e-04 - val_loss: 2.1673e-04
Epoch 252/350
31/31  0s 4ms/step - loss: 2.3320e-04 - val_loss: 2.1610e-04
Epoch 253/350
31/31  0s 4ms/step - loss: 2.2083e-04 - val_loss: 2.1599e-04
Epoch 254/350
31/31  0s 3ms/step - loss: 2.2383e-04 - val_loss: 2.1567e-04
Epoch 255/350
31/31  0s 4ms/step - loss: 2.3467e-04 - val_loss: 2.1551e-04
Epoch 256/350
31/31  0s 4ms/step - loss: 2.2593e-04 - val_loss: 2.1503e-04
Epoch 257/350
31/31  0s 4ms/step - loss: 2.1063e-04 - val_loss: 2.1464e-04
Epoch 258/350
31/31  0s 4ms/step - loss: 2.2670e-04 - val_loss: 2.1439e-04
Epoch 259/350
31/31  0s 3ms/step - loss: 2.1563e-04 - val_loss: 2.1422e-04
Epoch 260/350
31/31  0s 4ms/step - loss: 2.1897e-04 - val_loss: 2.1398e-04
Epoch 261/350
31/31  0s 4ms/step - loss: 2.4046e-04 - val_loss: 2.1374e-04
Epoch 262/350
31/31  0s 4ms/step - loss: 2.2263e-04 - val_loss: 2.1334e-04
Epoch 263/350
31/31  0s 4ms/step - loss: 2.1588e-04 - val_loss: 2.1305e-04
Epoch 264/350
31/31  0s 4ms/step - loss: 2.3484e-04 - val_loss: 2.1292e-04
Epoch 265/350
31/31  0s 4ms/step - loss: 2.1303e-04 - val_loss: 2.1268e-04
Epoch 266/350
31/31  0s 3ms/step - loss: 2.2309e-04 - val_loss: 2.1264e-04
Epoch 267/350
31/31  0s 5ms/step - loss: 2.1479e-04 - val_loss: 2.1212e-04
Epoch 268/350
31/31  0s 4ms/step - loss: 2.3818e-04 - val_loss: 2.1205e-04
Epoch 269/350
31/31  0s 4ms/step - loss: 2.2981e-04 - val_loss: 2.1164e-04
Epoch 270/350
31/31  0s 4ms/step - loss: 2.3156e-04 - val_loss: 2.1150e-04
```

```
Epoch 271/350
31/31  0s 4ms/step - loss: 2.3433e-04 - val_loss: 2.1120e-04
Epoch 272/350
31/31  0s 4ms/step - loss: 2.1463e-04 - val_loss: 2.1084e-04
Epoch 273/350
31/31  0s 4ms/step - loss: 2.1969e-04 - val_loss: 2.1084e-04
Epoch 274/350
31/31  0s 3ms/step - loss: 2.2105e-04 - val_loss: 2.1052e-04
Epoch 275/350
31/31  0s 4ms/step - loss: 2.0029e-04 - val_loss: 2.1020e-04
Epoch 276/350
31/31  0s 4ms/step - loss: 2.2881e-04 - val_loss: 2.1013e-04
Epoch 277/350
31/31  0s 4ms/step - loss: 2.2436e-04 - val_loss: 2.0973e-04
Epoch 278/350
31/31  0s 4ms/step - loss: 2.0763e-04 - val_loss: 2.0952e-04
Epoch 279/350
31/31  0s 4ms/step - loss: 2.1689e-04 - val_loss: 2.0933e-04
Epoch 280/350
31/31  0s 5ms/step - loss: 2.0525e-04 - val_loss: 2.0914e-04
Epoch 281/350
31/31  0s 3ms/step - loss: 2.0969e-04 - val_loss: 2.0886e-04
Epoch 282/350
31/31  0s 4ms/step - loss: 2.2953e-04 - val_loss: 2.0877e-04
Epoch 283/350
31/31  0s 4ms/step - loss: 2.1767e-04 - val_loss: 2.0853e-04
Epoch 284/350
31/31  0s 4ms/step - loss: 2.1847e-04 - val_loss: 2.0819e-04
Epoch 285/350
31/31  0s 4ms/step - loss: 2.1546e-04 - val_loss: 2.0811e-04
Epoch 286/350
31/31  0s 4ms/step - loss: 2.1574e-04 - val_loss: 2.0793e-04
Epoch 287/350
31/31  0s 4ms/step - loss: 2.0963e-04 - val_loss: 2.0764e-04
Epoch 288/350
31/31  0s 4ms/step - loss: 2.2068e-04 - val_loss: 2.0757e-04
Epoch 289/350
31/31  0s 4ms/step - loss: 1.9897e-04 - val_loss: 2.0738e-04
Epoch 290/350
31/31  0s 4ms/step - loss: 2.1572e-04 - val_loss: 2.0719e-04
Epoch 291/350
31/31  0s 4ms/step - loss: 2.1309e-04 - val_loss: 2.0697e-04
Epoch 292/350
31/31  0s 4ms/step - loss: 2.1309e-04 - val_loss: 2.0680e-04
Epoch 293/350
31/31  0s 3ms/step - loss: 2.2720e-04 - val_loss: 2.0671e-04
Epoch 294/350
31/31  0s 3ms/step - loss: 2.0990e-04 - val_loss: 2.0659e-04
Epoch 295/350
31/31  0s 4ms/step - loss: 2.0501e-04 - val_loss: 2.0619e-04
Epoch 296/350
31/31  0s 4ms/step - loss: 2.1179e-04 - val_loss: 2.0606e-04
Epoch 297/350
31/31  0s 4ms/step - loss: 2.1826e-04 - val_loss: 2.0585e-04
Epoch 298/350
31/31  0s 4ms/step - loss: 1.9870e-04 - val_loss: 2.0593e-04
Epoch 299/350
31/31  0s 4ms/step - loss: 2.2780e-04 - val_loss: 2.0575e-04
Epoch 300/350
31/31  0s 4ms/step - loss: 2.0097e-04 - val_loss: 2.0546e-04
```

```
Epoch 301/350
31/31  0s 4ms/step - loss: 2.2071e-04 - val_loss: 2.0521e-04
Epoch 302/350
31/31  0s 3ms/step - loss: 1.9862e-04 - val_loss: 2.0519e-04
Epoch 303/350
31/31  0s 4ms/step - loss: 2.0336e-04 - val_loss: 2.0494e-04
Epoch 304/350
31/31  0s 4ms/step - loss: 2.1712e-04 - val_loss: 2.0492e-04
Epoch 305/350
31/31  0s 4ms/step - loss: 2.0353e-04 - val_loss: 2.0463e-04
Epoch 306/350
31/31  0s 3ms/step - loss: 2.1409e-04 - val_loss: 2.0450e-04
Epoch 307/350
31/31  0s 4ms/step - loss: 2.1249e-04 - val_loss: 2.0449e-04
Epoch 308/350
31/31  0s 4ms/step - loss: 2.1514e-04 - val_loss: 2.0421e-04
Epoch 309/350
31/31  0s 4ms/step - loss: 2.1128e-04 - val_loss: 2.0394e-04
Epoch 310/350
31/31  0s 4ms/step - loss: 1.9921e-04 - val_loss: 2.0393e-04
Epoch 311/350
31/31  0s 4ms/step - loss: 2.2008e-04 - val_loss: 2.0366e-04
Epoch 312/350
31/31  0s 3ms/step - loss: 2.0216e-04 - val_loss: 2.0357e-04
Epoch 313/350
31/31  0s 4ms/step - loss: 2.0661e-04 - val_loss: 2.0337e-04
Epoch 314/350
31/31  0s 4ms/step - loss: 2.2240e-04 - val_loss: 2.0343e-04
Epoch 315/350
31/31  0s 3ms/step - loss: 2.0416e-04 - val_loss: 2.0314e-04
Epoch 316/350
31/31  0s 3ms/step - loss: 2.1321e-04 - val_loss: 2.0311e-04
Epoch 317/350
31/31  0s 4ms/step - loss: 2.1170e-04 - val_loss: 2.0304e-04
Epoch 318/350
31/31  0s 4ms/step - loss: 2.0606e-04 - val_loss: 2.0284e-04
Epoch 319/350
31/31  0s 4ms/step - loss: 2.1507e-04 - val_loss: 2.0258e-04
Epoch 320/350
31/31  0s 4ms/step - loss: 1.9794e-04 - val_loss: 2.0248e-04
Epoch 321/350
31/31  0s 4ms/step - loss: 2.0619e-04 - val_loss: 2.0238e-04
Epoch 322/350
31/31  0s 3ms/step - loss: 2.2175e-04 - val_loss: 2.0225e-04
Epoch 323/350
31/31  0s 4ms/step - loss: 2.1865e-04 - val_loss: 2.0215e-04
Epoch 324/350
31/31  0s 4ms/step - loss: 2.0982e-04 - val_loss: 2.0201e-04
Epoch 325/350
31/31  0s 4ms/step - loss: 2.1024e-04 - val_loss: 2.0201e-04
Epoch 326/350
31/31  0s 3ms/step - loss: 2.0268e-04 - val_loss: 2.0183e-04
Epoch 327/350
31/31  0s 4ms/step - loss: 1.9721e-04 - val_loss: 2.0176e-04
Epoch 328/350
31/31  0s 3ms/step - loss: 1.9514e-04 - val_loss: 2.0172e-04
Epoch 329/350
31/31  0s 4ms/step - loss: 2.0991e-04 - val_loss: 2.0149e-04
Epoch 330/350
31/31  0s 4ms/step - loss: 2.0969e-04 - val_loss: 2.0131e-04
```

```

Epoch 331/350
31/31  0s 3ms/step - loss: 2.0704e-04 - val_loss: 2.0117e-04
Epoch 332/350
31/31  0s 4ms/step - loss: 1.9152e-04 - val_loss: 2.0106e-04
Epoch 333/350
31/31  0s 4ms/step - loss: 2.0893e-04 - val_loss: 2.0106e-04
Epoch 334/350
31/31  0s 4ms/step - loss: 1.9503e-04 - val_loss: 2.0088e-04
Epoch 335/350
31/31  0s 3ms/step - loss: 2.0499e-04 - val_loss: 2.0079e-04
Epoch 336/350
31/31  0s 4ms/step - loss: 2.0441e-04 - val_loss: 2.0082e-04
Epoch 337/350
31/31  0s 4ms/step - loss: 2.1289e-04 - val_loss: 2.0060e-04
Epoch 338/350
31/31  0s 4ms/step - loss: 2.1388e-04 - val_loss: 2.0054e-04
Epoch 339/350
31/31  0s 4ms/step - loss: 2.0253e-04 - val_loss: 2.0033e-04
Epoch 340/350
31/31  0s 4ms/step - loss: 2.2594e-04 - val_loss: 2.0024e-04
Epoch 341/350
31/31  0s 4ms/step - loss: 1.9862e-04 - val_loss: 2.0020e-04
Epoch 342/350
31/31  0s 4ms/step - loss: 1.9603e-04 - val_loss: 2.0020e-04
Epoch 343/350
31/31  0s 4ms/step - loss: 2.0171e-04 - val_loss: 2.0010e-04
Epoch 344/350
31/31  0s 4ms/step - loss: 2.1142e-04 - val_loss: 1.9984e-04
Epoch 345/350
31/31  0s 4ms/step - loss: 2.0991e-04 - val_loss: 1.9982e-04
Epoch 346/350
31/31  0s 4ms/step - loss: 1.9801e-04 - val_loss: 1.9965e-04
Epoch 347/350
31/31  0s 4ms/step - loss: 2.0323e-04 - val_loss: 1.9969e-04
Epoch 348/350
31/31  0s 4ms/step - loss: 2.1097e-04 - val_loss: 1.9958e-04
Epoch 349/350
31/31  0s 3ms/step - loss: 1.8777e-04 - val_loss: 1.9949e-04
Epoch 350/350
31/31  0s 4ms/step - loss: 1.9985e-04 - val_loss: 1.9914e-04

```

```
Out[12]: <keras.src.callbacks.history.History at 0x7968caeea010>
```

We can analyze the performance of the linear autoencoder by extracting the reconstruction error of the model in the training and test samples. We compute, similar to computing R^2 in regression settings, the proportion of the variance of the inputs that is left unexplained by the model.

Remember that the lower bound on the in-sample reconstruction error, using a linear autoencoder and an MSE loss, is set by the sum of the principal eigenvalues $k+1, \dots, n$. From above, we obtained that the first three principal components explain roughly 67% of the variation in the data. The trained autoencoder has a reconstruction error of 32%, meaning that the error is close to the minimum possible when using a three-dimensional encoding.

Importantly, notice that we are using a validation sample with an early stopping criterion to avoid overfitting. This means that training may stop before the lower bound is

achieved, so we should not necessarily reach the PCA's minimum reconstruction error using the trained autoencoder.

Below, we report the reconstruction error of the model in the training set and validation set separately. The results depend on the random seed that initializes the training algorithm. However, after several runs, the model should display a relatively good out-of-sample performance in terms of reconstruction error.

```
In [13]: Y_pred = model.predict(Y_train, verbose=0)
error = np.sum((Y_train - Y_pred) ** 2) / np.sum((Y_train - np.mean(Y_train)) ** 2)
print("Reconstruction error in training sample", error)
Y_pred = model.predict(Y_test, verbose=0)
error_oos = np.sum((Y_test - Y_pred) ** 2) / np.sum((Y_test - np.mean(Y_test)) ** 2)
print("Reconstruction error in test sample", error_oos)
```

Reconstruction error in training sample 0.3179451109621889

Reconstruction error in test sample 0.4889370196068262

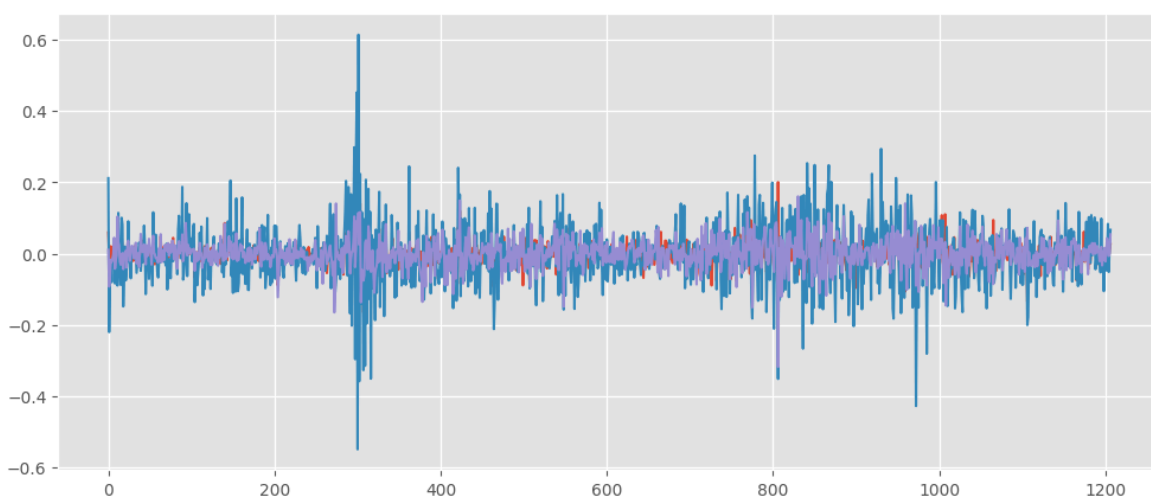
We can recover the time series of each latent factor generated by the autoencoder.

Notice that the latent factors recovered by the autoencoder are correlated, as opposed to the PCA factors. Besides, the autoencoder factors are not ordered in terms of total variation.

```
In [14]: factors = model.encoder(Y_train).numpy()
factors_test = model.encoder(Y_test).numpy()
factors_cov = np.cov(factors, rowvar=False)
print(factors_cov)
```

```
[[0.00104645 0.00133243 0.0003107 ]
 [0.00133243 0.00750614 0.00162256]
 [0.0003107  0.00162256 0.00160836]]
```

```
In [15]: # Plot the evolution of latent factors in the training period
plt.figure(figsize=(12, 5))
plt.plot(factors)
plt.show()
```

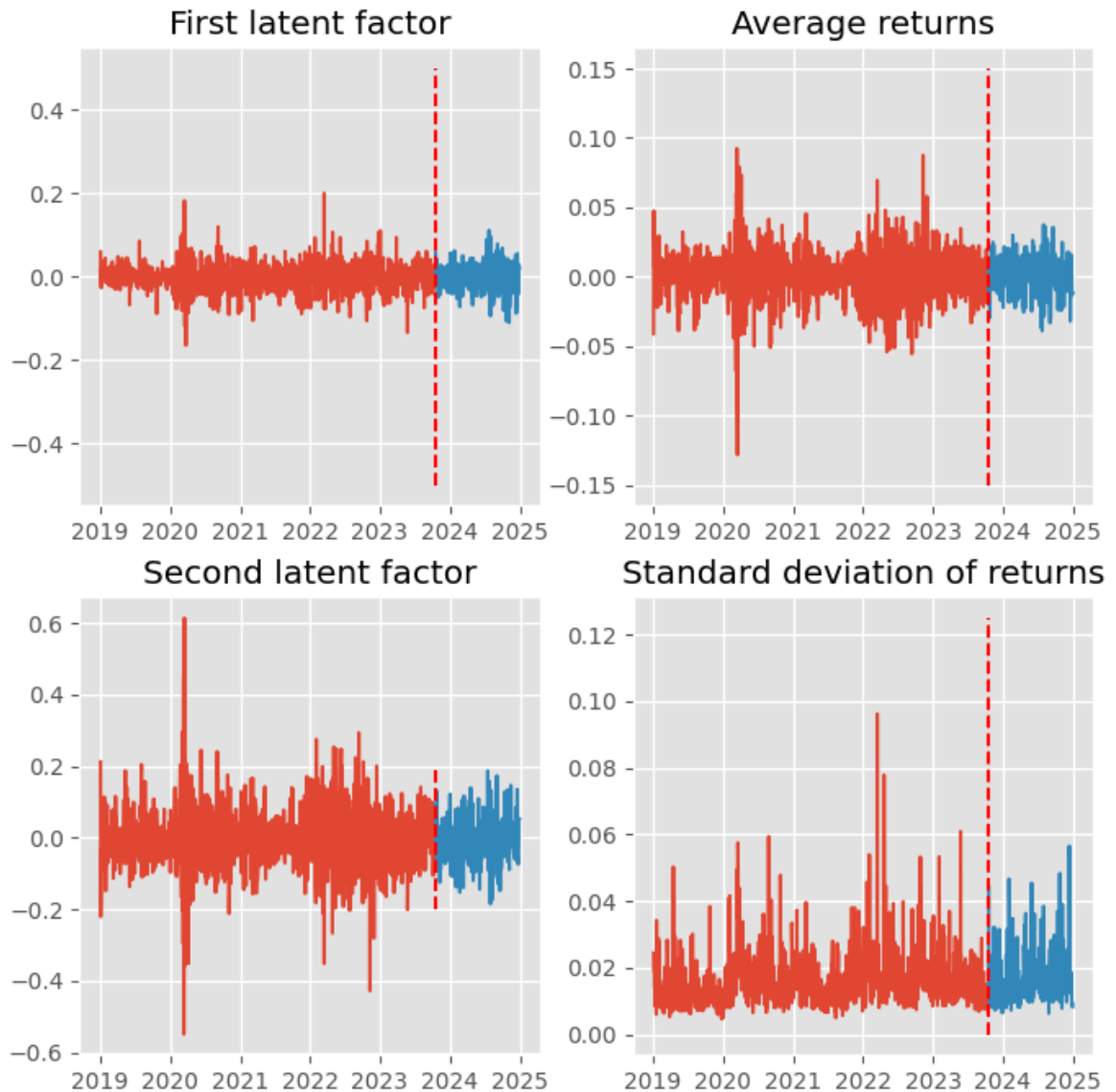


Although the results may vary depending on the seed, the highest-variance latent factor recovered by the autoencoder tends to capture the average level of returns across the stocks considered. The second latent factor in terms of variance seems to capture

variations in the cross-sectional standard deviation of returns. That is, when the second factor takes a high value it seems that returns across stocks are more disperse.

Note: The red dashed line denotes the end of the training sample.

```
In [16]: fig, axs = plt.subplots(2, 2, figsize=(8, 8))
# First factor
axs[0, 0].plot(df.index[: train_size + 1], factors[:, 0])
axs[0, 0].plot(df.index[train_size + 1 :], factors_test[:, 0])
axs[0, 0].plot([df.index[train_size + 1], df.index[train_size + 1]], [-0.5, 0.5])
axs[0, 0].title.set_text("First latent factor")
# Average of returns
axs[0, 1].plot(df.index[: train_size + 1], np.mean(Y_train, axis=1))
axs[0, 1].plot(df.index[train_size + 1 :], np.mean(Y_test, axis=1))
axs[0, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [-0.15, 0.15], "r--"
)
axs[0, 1].title.set_text("Average returns")
# Second factor
axs[1, 0].plot(df.index[: train_size + 1], factors[:, 1])
axs[1, 0].plot(df.index[train_size + 1 :], factors_test[:, 1])
axs[1, 0].plot([df.index[train_size + 1], df.index[train_size + 1]], [-0.2, 0.2])
axs[1, 0].title.set_text("Second latent factor")
# Standard deviation of returns
axs[1, 1].plot(df.index[: train_size + 1], np.std(Y_train, axis=1))
axs[1, 1].plot(df.index[train_size + 1 :], np.std(Y_test, axis=1))
axs[1, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [0.0, 0.125], "r--"
)
axs[1, 1].title.set_text("Standard deviation of returns")
```



How is each stock in our sample exposed to each of the latent factors extracted by the autoencoder? This exposure is measured by each stock's loadings (the trainable weights in the output layer). Below, we generate scatter plots that display the relationship between the loadings of each stock on each pair of factors.

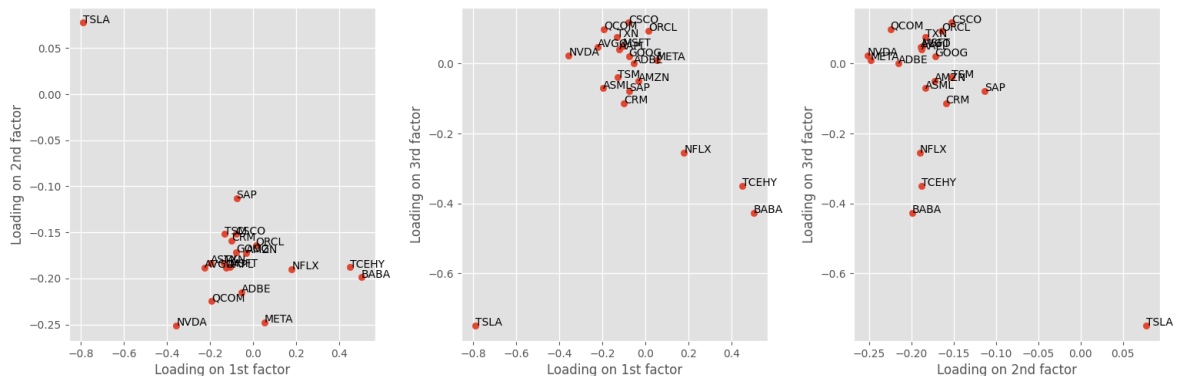
Noticeably, Tesla represents a clear outlier, a results that tends to appear regardless of the random seed that is used in the training algorithm. This suggests that a large portion of the cross-sectional variation in stock returns is actually driven by the behavior of Tesla's stock price. In other words, Tesla's stock price displays a behavior of its own that is separate from other stocks.

```
In [17]: # Extract Loadings generated by the autoencoder
w_decoder = model.decoder.layers[0].get_weights()[0]
fig, axs = plt.subplots(1, 3, figsize=(16, 6))
fig.tight_layout(pad=5.0)
axs[0].scatter(w_decoder[0], w_decoder[1])
axs[0].set_ylabel("Loading on 2nd factor")
axs[0].set_xlabel("Loading on 1st factor")
axs[1].scatter(w_decoder[0], w_decoder[2])
axs[1].set_ylabel("Loading on 3rd factor")
axs[1].set_xlabel("Loading on 1st factor")
axs[2].scatter(w_decoder[1], w_decoder[2])
```

```

axs[2].set_ylabel("Loading on 3rd factor")
axs[2].set_xlabel("Loading on 2nd factor")
for i, txt in enumerate(tickers):
    axs[0].annotate(txt, (w_decoder[0, i], w_decoder[1, i]))
    axs[1].annotate(txt, (w_decoder[0, i], w_decoder[2, i]))
    axs[2].annotate(txt, (w_decoder[1, i], w_decoder[2, i]))

```



4. AN AUTOENCODER WITH NON-LINEAR ACTIVATIONS

In the blocks of code below we build and train an autoencoder that includes two additional hidden layers, one on each side of the bottleneck, with non-linear activations. The trained model displays slightly less reconstruction error than the linear benchmark in the training sample, although the error in the test sample is approximately the same.

The non-linear autoencoder may be able to capture richer features than the linear version. However, it becomes harder to grasp the type of variation that each latent factor is recovering from the data. As we show below in the figures, the factors are not so tightly related to average returns or dispersion, while the presence of an intermediate hidden layer makes it hard to grasp the exposure of each stock to the latent factors. Thus, adding non-linearities and hidden layers may enhance the explanatory ability of our models, but we likely lose a great deal of interpretability.

```

In [18]: class autoencoder_nonlinear(tf.keras.Model):
    def __init__(self, output_dim, bottleneck_dim=1):
        super(autoencoder_nonlinear, self).__init__()
        self.output_dim = output_dim
        self.bottleneck_dim = bottleneck_dim
        act_fun = "relu"
        self.encoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(12, activation=act_fun),
                tf.keras.layers.Dense(bottleneck_dim),
            ]
        )
        self.decoder = tf.keras.Sequential(
            [
                tf.keras.layers.Dense(12, activation=act_fun),
                tf.keras.layers.Dense(output_dim),
            ]
        )

    def call(self, x):

```

```
        encoded = self.encoder(x)
        decoded = self.decoder(encoded)
    return decoded
```

```
In [19]: nlatent = 3
# to make this notebook's output stable
# np.random.seed(42)
# tf.random.set_seed(42)
val_split = 0.2
early_stop = tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=50)
model_nonlinear = autoencoder_nonlinear(Y_train.shape[1], nlatent)
adam = tf.keras.optimizers.Adam(learning_rate=1e-4)
model_nonlinear.compile(optimizer=adam, loss="mse")
model_nonlinear.fit(
    x=Y_train,
    y=Y_train,
    batch_size=32,
    epochs=350,
    validation_split=val_split,
    callbacks=[early_stop],
    verbose=1,
)
```

```
Epoch 1/350
31/31  1s 8ms/step - loss: 7.6684e-04 - val_loss: 5.6887e-04
Epoch 2/350
31/31  0s 4ms/step - loss: 6.7629e-04 - val_loss: 5.6141e-04
Epoch 3/350
31/31  0s 4ms/step - loss: 6.2947e-04 - val_loss: 5.5522e-04
Epoch 4/350
31/31  0s 4ms/step - loss: 6.3512e-04 - val_loss: 5.4929e-04
Epoch 5/350
31/31  0s 4ms/step - loss: 7.1331e-04 - val_loss: 5.4184e-04
Epoch 6/350
31/31  0s 4ms/step - loss: 6.2695e-04 - val_loss: 5.3452e-04
Epoch 7/350
31/31  0s 4ms/step - loss: 6.1891e-04 - val_loss: 5.2715e-04
Epoch 8/350
31/31  0s 4ms/step - loss: 5.8189e-04 - val_loss: 5.1989e-04
Epoch 9/350
31/31  0s 3ms/step - loss: 6.3155e-04 - val_loss: 5.1103e-04
Epoch 10/350
31/31  0s 4ms/step - loss: 6.0388e-04 - val_loss: 5.0307e-04
Epoch 11/350
31/31  0s 4ms/step - loss: 6.0035e-04 - val_loss: 4.9376e-04
Epoch 12/350
31/31  0s 4ms/step - loss: 6.0533e-04 - val_loss: 4.8527e-04
Epoch 13/350
31/31  0s 4ms/step - loss: 5.7830e-04 - val_loss: 4.7579e-04
Epoch 14/350
31/31  0s 4ms/step - loss: 5.5804e-04 - val_loss: 4.6718e-04
Epoch 15/350
31/31  0s 4ms/step - loss: 5.6468e-04 - val_loss: 4.5852e-04
Epoch 16/350
31/31  0s 3ms/step - loss: 5.5596e-04 - val_loss: 4.5038e-04
Epoch 17/350
31/31  0s 4ms/step - loss: 5.2570e-04 - val_loss: 4.4298e-04
Epoch 18/350
31/31  0s 4ms/step - loss: 5.4704e-04 - val_loss: 4.3462e-04
Epoch 19/350
31/31  0s 4ms/step - loss: 5.0911e-04 - val_loss: 4.2834e-04
Epoch 20/350
31/31  0s 4ms/step - loss: 4.6692e-04 - val_loss: 4.2099e-04
Epoch 21/350
31/31  0s 4ms/step - loss: 4.7658e-04 - val_loss: 4.1451e-04
Epoch 22/350
31/31  0s 3ms/step - loss: 5.0085e-04 - val_loss: 4.0858e-04
Epoch 23/350
31/31  0s 4ms/step - loss: 4.9184e-04 - val_loss: 4.0117e-04
Epoch 24/350
31/31  0s 4ms/step - loss: 4.7263e-04 - val_loss: 3.9622e-04
Epoch 25/350
31/31  0s 4ms/step - loss: 4.4502e-04 - val_loss: 3.9005e-04
Epoch 26/350
31/31  0s 4ms/step - loss: 4.5332e-04 - val_loss: 3.8408e-04
Epoch 27/350
31/31  0s 4ms/step - loss: 4.4688e-04 - val_loss: 3.7942e-04
Epoch 28/350
31/31  0s 4ms/step - loss: 4.1738e-04 - val_loss: 3.7461e-04
Epoch 29/350
31/31  0s 3ms/step - loss: 4.2692e-04 - val_loss: 3.7044e-04
Epoch 30/350
31/31  0s 4ms/step - loss: 4.4853e-04 - val_loss: 3.6541e-04
```

```
Epoch 31/350
31/31  0s 4ms/step - loss: 3.8681e-04 - val_loss: 3.6099e-04
Epoch 32/350
31/31  0s 4ms/step - loss: 3.9449e-04 - val_loss: 3.5782e-04
Epoch 33/350
31/31  0s 4ms/step - loss: 3.8867e-04 - val_loss: 3.5331e-04
Epoch 34/350
31/31  0s 4ms/step - loss: 3.9347e-04 - val_loss: 3.4975e-04
Epoch 35/350
31/31  0s 4ms/step - loss: 3.8846e-04 - val_loss: 3.4636e-04
Epoch 36/350
31/31  0s 4ms/step - loss: 3.9032e-04 - val_loss: 3.4285e-04
Epoch 37/350
31/31  0s 4ms/step - loss: 3.8907e-04 - val_loss: 3.4014e-04
Epoch 38/350
31/31  0s 4ms/step - loss: 3.9369e-04 - val_loss: 3.3724e-04
Epoch 39/350
31/31  0s 4ms/step - loss: 3.5860e-04 - val_loss: 3.3363e-04
Epoch 40/350
31/31  0s 4ms/step - loss: 3.4960e-04 - val_loss: 3.3139e-04
Epoch 41/350
31/31  0s 6ms/step - loss: 3.9924e-04 - val_loss: 3.2752e-04
Epoch 42/350
31/31  0s 6ms/step - loss: 3.5631e-04 - val_loss: 3.2494e-04
Epoch 43/350
31/31  0s 6ms/step - loss: 3.3630e-04 - val_loss: 3.2365e-04
Epoch 44/350
31/31  0s 6ms/step - loss: 3.5517e-04 - val_loss: 3.2041e-04
Epoch 45/350
31/31  0s 6ms/step - loss: 3.2983e-04 - val_loss: 3.1884e-04
Epoch 46/350
31/31  0s 5ms/step - loss: 3.5246e-04 - val_loss: 3.1634e-04
Epoch 47/350
31/31  0s 6ms/step - loss: 3.5616e-04 - val_loss: 3.1338e-04
Epoch 48/350
31/31  0s 3ms/step - loss: 3.4471e-04 - val_loss: 3.1196e-04
Epoch 49/350
31/31  0s 4ms/step - loss: 3.4331e-04 - val_loss: 3.1018e-04
Epoch 50/350
31/31  0s 4ms/step - loss: 3.3593e-04 - val_loss: 3.0859e-04
Epoch 51/350
31/31  0s 3ms/step - loss: 3.2970e-04 - val_loss: 3.0693e-04
Epoch 52/350
31/31  0s 5ms/step - loss: 3.3326e-04 - val_loss: 3.0488e-04
Epoch 53/350
31/31  0s 5ms/step - loss: 3.2406e-04 - val_loss: 3.0272e-04
Epoch 54/350
31/31  0s 5ms/step - loss: 3.2708e-04 - val_loss: 3.0145e-04
Epoch 55/350
31/31  0s 5ms/step - loss: 3.2104e-04 - val_loss: 3.0047e-04
Epoch 56/350
31/31  0s 3ms/step - loss: 3.3432e-04 - val_loss: 2.9948e-04
Epoch 57/350
31/31  0s 4ms/step - loss: 3.3435e-04 - val_loss: 2.9625e-04
Epoch 58/350
31/31  0s 4ms/step - loss: 3.3143e-04 - val_loss: 2.9559e-04
Epoch 59/350
31/31  0s 4ms/step - loss: 3.5125e-04 - val_loss: 2.9447e-04
Epoch 60/350
31/31  0s 4ms/step - loss: 3.0695e-04 - val_loss: 2.9338e-04
```

```
Epoch 61/350
31/31  0s 4ms/step - loss: 3.0815e-04 - val_loss: 2.9146e-04
Epoch 62/350
31/31  0s 4ms/step - loss: 3.0087e-04 - val_loss: 2.9084e-04
Epoch 63/350
31/31  0s 4ms/step - loss: 3.2577e-04 - val_loss: 2.8957e-04
Epoch 64/350
31/31  0s 4ms/step - loss: 3.2133e-04 - val_loss: 2.8841e-04
Epoch 65/350
31/31  0s 6ms/step - loss: 3.0689e-04 - val_loss: 2.8729e-04
Epoch 66/350
31/31  0s 4ms/step - loss: 2.9215e-04 - val_loss: 2.8597e-04
Epoch 67/350
31/31  0s 4ms/step - loss: 3.2651e-04 - val_loss: 2.8513e-04
Epoch 68/350
31/31  0s 4ms/step - loss: 3.0124e-04 - val_loss: 2.8415e-04
Epoch 69/350
31/31  0s 4ms/step - loss: 3.1816e-04 - val_loss: 2.8278e-04
Epoch 70/350
31/31  0s 4ms/step - loss: 2.8879e-04 - val_loss: 2.8250e-04
Epoch 71/350
31/31  0s 4ms/step - loss: 3.0999e-04 - val_loss: 2.8091e-04
Epoch 72/350
31/31  0s 4ms/step - loss: 2.8722e-04 - val_loss: 2.7955e-04
Epoch 73/350
31/31  0s 4ms/step - loss: 2.9748e-04 - val_loss: 2.7961e-04
Epoch 74/350
31/31  0s 4ms/step - loss: 3.1717e-04 - val_loss: 2.7781e-04
Epoch 75/350
31/31  0s 4ms/step - loss: 3.1093e-04 - val_loss: 2.7685e-04
Epoch 76/350
31/31  0s 4ms/step - loss: 2.8151e-04 - val_loss: 2.7589e-04
Epoch 77/350
31/31  0s 4ms/step - loss: 2.6536e-04 - val_loss: 2.7607e-04
Epoch 78/350
31/31  0s 4ms/step - loss: 2.6969e-04 - val_loss: 2.7446e-04
Epoch 79/350
31/31  0s 4ms/step - loss: 3.0753e-04 - val_loss: 2.7360e-04
Epoch 80/350
31/31  0s 4ms/step - loss: 2.9730e-04 - val_loss: 2.7245e-04
Epoch 81/350
31/31  0s 4ms/step - loss: 3.1307e-04 - val_loss: 2.7207e-04
Epoch 82/350
31/31  0s 4ms/step - loss: 3.1436e-04 - val_loss: 2.7078e-04
Epoch 83/350
31/31  0s 3ms/step - loss: 2.6424e-04 - val_loss: 2.7041e-04
Epoch 84/350
31/31  0s 4ms/step - loss: 2.8041e-04 - val_loss: 2.6943e-04
Epoch 85/350
31/31  0s 5ms/step - loss: 3.0792e-04 - val_loss: 2.6865e-04
Epoch 86/350
31/31  0s 6ms/step - loss: 2.8270e-04 - val_loss: 2.6755e-04
Epoch 87/350
31/31  0s 5ms/step - loss: 2.8223e-04 - val_loss: 2.6747e-04
Epoch 88/350
31/31  0s 4ms/step - loss: 2.6932e-04 - val_loss: 2.6609e-04
Epoch 89/350
31/31  0s 4ms/step - loss: 2.6900e-04 - val_loss: 2.6555e-04
Epoch 90/350
31/31  0s 4ms/step - loss: 2.9153e-04 - val_loss: 2.6506e-04
```

```
Epoch 91/350
31/31  0s 4ms/step - loss: 2.7369e-04 - val_loss: 2.6403e-04
Epoch 92/350
31/31  0s 4ms/step - loss: 2.8406e-04 - val_loss: 2.6378e-04
Epoch 93/350
31/31  0s 4ms/step - loss: 2.8459e-04 - val_loss: 2.6255e-04
Epoch 94/350
31/31  0s 4ms/step - loss: 2.6245e-04 - val_loss: 2.6210e-04
Epoch 95/350
31/31  0s 4ms/step - loss: 2.7069e-04 - val_loss: 2.6169e-04
Epoch 96/350
31/31  0s 4ms/step - loss: 2.7910e-04 - val_loss: 2.6110e-04
Epoch 97/350
31/31  0s 4ms/step - loss: 2.6897e-04 - val_loss: 2.5982e-04
Epoch 98/350
31/31  0s 3ms/step - loss: 2.7645e-04 - val_loss: 2.5920e-04
Epoch 99/350
31/31  0s 5ms/step - loss: 2.8929e-04 - val_loss: 2.5857e-04
Epoch 100/350
31/31  0s 4ms/step - loss: 2.6989e-04 - val_loss: 2.5787e-04
Epoch 101/350
31/31  0s 3ms/step - loss: 2.7571e-04 - val_loss: 2.5751e-04
Epoch 102/350
31/31  0s 4ms/step - loss: 2.8700e-04 - val_loss: 2.5659e-04
Epoch 103/350
31/31  0s 4ms/step - loss: 2.8108e-04 - val_loss: 2.5594e-04
Epoch 104/350
31/31  0s 4ms/step - loss: 2.5291e-04 - val_loss: 2.5524e-04
Epoch 105/350
31/31  0s 4ms/step - loss: 2.8573e-04 - val_loss: 2.5398e-04
Epoch 106/350
31/31  0s 4ms/step - loss: 2.4930e-04 - val_loss: 2.5386e-04
Epoch 107/350
31/31  0s 4ms/step - loss: 2.7561e-04 - val_loss: 2.5356e-04
Epoch 108/350
31/31  0s 4ms/step - loss: 2.8838e-04 - val_loss: 2.5242e-04
Epoch 109/350
31/31  0s 4ms/step - loss: 2.5744e-04 - val_loss: 2.5138e-04
Epoch 110/350
31/31  0s 4ms/step - loss: 2.7010e-04 - val_loss: 2.5086e-04
Epoch 111/350
31/31  0s 4ms/step - loss: 2.7774e-04 - val_loss: 2.4990e-04
Epoch 112/350
31/31  0s 4ms/step - loss: 2.5348e-04 - val_loss: 2.4928e-04
Epoch 113/350
31/31  0s 3ms/step - loss: 2.6336e-04 - val_loss: 2.4857e-04
Epoch 114/350
31/31  0s 4ms/step - loss: 2.4283e-04 - val_loss: 2.4824e-04
Epoch 115/350
31/31  0s 4ms/step - loss: 2.7206e-04 - val_loss: 2.4728e-04
Epoch 116/350
31/31  0s 4ms/step - loss: 2.6939e-04 - val_loss: 2.4638e-04
Epoch 117/350
31/31  0s 4ms/step - loss: 2.5857e-04 - val_loss: 2.4606e-04
Epoch 118/350
31/31  0s 4ms/step - loss: 2.7000e-04 - val_loss: 2.4530e-04
Epoch 119/350
31/31  0s 3ms/step - loss: 2.3783e-04 - val_loss: 2.4441e-04
Epoch 120/350
31/31  0s 4ms/step - loss: 2.6705e-04 - val_loss: 2.4402e-04
```

```
Epoch 121/350
31/31  0s 4ms/step - loss: 2.6339e-04 - val_loss: 2.4299e-04
Epoch 122/350
31/31  0s 4ms/step - loss: 2.6074e-04 - val_loss: 2.4203e-04
Epoch 123/350
31/31  0s 4ms/step - loss: 2.5216e-04 - val_loss: 2.4129e-04
Epoch 124/350
31/31  0s 4ms/step - loss: 2.3659e-04 - val_loss: 2.4069e-04
Epoch 125/350
31/31  0s 4ms/step - loss: 2.5897e-04 - val_loss: 2.3959e-04
Epoch 126/350
31/31  0s 4ms/step - loss: 2.5349e-04 - val_loss: 2.3883e-04
Epoch 127/350
31/31  0s 4ms/step - loss: 2.4613e-04 - val_loss: 2.3795e-04
Epoch 128/350
31/31  0s 4ms/step - loss: 2.6000e-04 - val_loss: 2.3687e-04
Epoch 129/350
31/31  0s 4ms/step - loss: 2.3346e-04 - val_loss: 2.3625e-04
Epoch 130/350
31/31  0s 4ms/step - loss: 2.5746e-04 - val_loss: 2.3553e-04
Epoch 131/350
31/31  0s 4ms/step - loss: 2.5393e-04 - val_loss: 2.3473e-04
Epoch 132/350
31/31  0s 4ms/step - loss: 2.6801e-04 - val_loss: 2.3370e-04
Epoch 133/350
31/31  0s 3ms/step - loss: 2.4719e-04 - val_loss: 2.3242e-04
Epoch 134/350
31/31  0s 4ms/step - loss: 2.3875e-04 - val_loss: 2.3209e-04
Epoch 135/350
31/31  0s 4ms/step - loss: 2.2582e-04 - val_loss: 2.3093e-04
Epoch 136/350
31/31  0s 4ms/step - loss: 2.4557e-04 - val_loss: 2.3052e-04
Epoch 137/350
31/31  0s 5ms/step - loss: 2.5404e-04 - val_loss: 2.2927e-04
Epoch 138/350
31/31  0s 3ms/step - loss: 2.7054e-04 - val_loss: 2.2855e-04
Epoch 139/350
31/31  0s 3ms/step - loss: 2.5051e-04 - val_loss: 2.2799e-04
Epoch 140/350
31/31  0s 4ms/step - loss: 2.4993e-04 - val_loss: 2.2683e-04
Epoch 141/350
31/31  0s 4ms/step - loss: 2.5848e-04 - val_loss: 2.2593e-04
Epoch 142/350
31/31  0s 4ms/step - loss: 2.2620e-04 - val_loss: 2.2545e-04
Epoch 143/350
31/31  0s 4ms/step - loss: 2.4727e-04 - val_loss: 2.2433e-04
Epoch 144/350
31/31  0s 4ms/step - loss: 2.3988e-04 - val_loss: 2.2371e-04
Epoch 145/350
31/31  0s 4ms/step - loss: 2.6667e-04 - val_loss: 2.2333e-04
Epoch 146/350
31/31  0s 5ms/step - loss: 2.3766e-04 - val_loss: 2.2209e-04
Epoch 147/350
31/31  0s 4ms/step - loss: 2.3444e-04 - val_loss: 2.2155e-04
Epoch 148/350
31/31  0s 4ms/step - loss: 2.1763e-04 - val_loss: 2.2096e-04
Epoch 149/350
31/31  0s 4ms/step - loss: 2.3864e-04 - val_loss: 2.1989e-04
Epoch 150/350
31/31  0s 4ms/step - loss: 2.3049e-04 - val_loss: 2.1913e-04
```

```
Epoch 151/350
31/31  0s 4ms/step - loss: 2.2448e-04 - val_loss: 2.1825e-04
Epoch 152/350
31/31  0s 4ms/step - loss: 2.5715e-04 - val_loss: 2.1774e-04
Epoch 153/350
31/31  0s 4ms/step - loss: 2.2150e-04 - val_loss: 2.1705e-04
Epoch 154/350
31/31  0s 4ms/step - loss: 2.2644e-04 - val_loss: 2.1627e-04
Epoch 155/350
31/31  0s 4ms/step - loss: 2.2553e-04 - val_loss: 2.1573e-04
Epoch 156/350
31/31  0s 4ms/step - loss: 2.3170e-04 - val_loss: 2.1511e-04
Epoch 157/350
31/31  0s 4ms/step - loss: 2.4861e-04 - val_loss: 2.1423e-04
Epoch 158/350
31/31  0s 4ms/step - loss: 2.2992e-04 - val_loss: 2.1336e-04
Epoch 159/350
31/31  0s 3ms/step - loss: 2.3305e-04 - val_loss: 2.1322e-04
Epoch 160/350
31/31  0s 4ms/step - loss: 2.3245e-04 - val_loss: 2.1255e-04
Epoch 161/350
31/31  0s 4ms/step - loss: 2.3323e-04 - val_loss: 2.1236e-04
Epoch 162/350
31/31  0s 4ms/step - loss: 2.1822e-04 - val_loss: 2.1148e-04
Epoch 163/350
31/31  0s 4ms/step - loss: 2.1644e-04 - val_loss: 2.1089e-04
Epoch 164/350
31/31  0s 4ms/step - loss: 2.0697e-04 - val_loss: 2.1060e-04
Epoch 165/350
31/31  0s 4ms/step - loss: 2.1217e-04 - val_loss: 2.0971e-04
Epoch 166/350
31/31  0s 4ms/step - loss: 2.2394e-04 - val_loss: 2.0958e-04
Epoch 167/350
31/31  0s 4ms/step - loss: 2.1272e-04 - val_loss: 2.0917e-04
Epoch 168/350
31/31  0s 4ms/step - loss: 2.1337e-04 - val_loss: 2.0839e-04
Epoch 169/350
31/31  0s 4ms/step - loss: 2.2021e-04 - val_loss: 2.0805e-04
Epoch 170/350
31/31  0s 4ms/step - loss: 2.2408e-04 - val_loss: 2.0739e-04
Epoch 171/350
31/31  0s 4ms/step - loss: 2.0560e-04 - val_loss: 2.0678e-04
Epoch 172/350
31/31  0s 4ms/step - loss: 2.3028e-04 - val_loss: 2.0696e-04
Epoch 173/350
31/31  0s 4ms/step - loss: 2.2943e-04 - val_loss: 2.0662e-04
Epoch 174/350
31/31  0s 4ms/step - loss: 2.0895e-04 - val_loss: 2.0614e-04
Epoch 175/350
31/31  0s 4ms/step - loss: 2.3674e-04 - val_loss: 2.0590e-04
Epoch 176/350
31/31  0s 4ms/step - loss: 2.3880e-04 - val_loss: 2.0478e-04
Epoch 177/350
31/31  0s 4ms/step - loss: 2.0375e-04 - val_loss: 2.0523e-04
Epoch 178/350
31/31  0s 4ms/step - loss: 2.5151e-04 - val_loss: 2.0448e-04
Epoch 179/350
31/31  0s 4ms/step - loss: 2.2004e-04 - val_loss: 2.0424e-04
Epoch 180/350
31/31  0s 4ms/step - loss: 2.1041e-04 - val_loss: 2.0353e-04
```

```
Epoch 181/350
31/31  0s 3ms/step - loss: 2.1078e-04 - val_loss: 2.0353e-04
Epoch 182/350
31/31  0s 5ms/step - loss: 2.0853e-04 - val_loss: 2.0333e-04
Epoch 183/350
31/31  0s 3ms/step - loss: 2.1251e-04 - val_loss: 2.0316e-04
Epoch 184/350
31/31  0s 4ms/step - loss: 2.2410e-04 - val_loss: 2.0262e-04
Epoch 185/350
31/31  0s 4ms/step - loss: 2.1171e-04 - val_loss: 2.0160e-04
Epoch 186/350
31/31  0s 4ms/step - loss: 2.1867e-04 - val_loss: 2.0206e-04
Epoch 187/350
31/31  0s 3ms/step - loss: 2.4135e-04 - val_loss: 2.0151e-04
Epoch 188/350
31/31  0s 4ms/step - loss: 2.1081e-04 - val_loss: 2.0120e-04
Epoch 189/350
31/31  0s 3ms/step - loss: 2.3482e-04 - val_loss: 2.0101e-04
Epoch 190/350
31/31  0s 4ms/step - loss: 2.4107e-04 - val_loss: 2.0070e-04
Epoch 191/350
31/31  0s 4ms/step - loss: 2.0674e-04 - val_loss: 2.0036e-04
Epoch 192/350
31/31  0s 4ms/step - loss: 2.0915e-04 - val_loss: 2.0060e-04
Epoch 193/350
31/31  0s 4ms/step - loss: 2.1824e-04 - val_loss: 2.0027e-04
Epoch 194/350
31/31  0s 4ms/step - loss: 2.0515e-04 - val_loss: 1.9937e-04
Epoch 195/350
31/31  0s 4ms/step - loss: 1.9874e-04 - val_loss: 1.9950e-04
Epoch 196/350
31/31  0s 4ms/step - loss: 2.0570e-04 - val_loss: 1.9918e-04
Epoch 197/350
31/31  0s 4ms/step - loss: 2.1135e-04 - val_loss: 1.9897e-04
Epoch 198/350
31/31  0s 4ms/step - loss: 2.0553e-04 - val_loss: 1.9887e-04
Epoch 199/350
31/31  0s 4ms/step - loss: 2.2948e-04 - val_loss: 1.9851e-04
Epoch 200/350
31/31  0s 4ms/step - loss: 2.1294e-04 - val_loss: 1.9835e-04
Epoch 201/350
31/31  0s 5ms/step - loss: 2.1422e-04 - val_loss: 1.9769e-04
Epoch 202/350
31/31  0s 4ms/step - loss: 2.1610e-04 - val_loss: 1.9801e-04
Epoch 203/350
31/31  0s 4ms/step - loss: 2.1306e-04 - val_loss: 1.9771e-04
Epoch 204/350
31/31  0s 4ms/step - loss: 2.1151e-04 - val_loss: 1.9732e-04
Epoch 205/350
31/31  0s 3ms/step - loss: 2.1114e-04 - val_loss: 1.9735e-04
Epoch 206/350
31/31  0s 4ms/step - loss: 2.1810e-04 - val_loss: 1.9716e-04
Epoch 207/350
31/31  0s 4ms/step - loss: 2.3451e-04 - val_loss: 1.9690e-04
Epoch 208/350
31/31  0s 4ms/step - loss: 2.0838e-04 - val_loss: 1.9668e-04
Epoch 209/350
31/31  0s 4ms/step - loss: 2.0958e-04 - val_loss: 1.9666e-04
Epoch 210/350
31/31  0s 3ms/step - loss: 2.1117e-04 - val_loss: 1.9639e-04
```

```
Epoch 211/350
31/31  0s 4ms/step - loss: 2.0318e-04 - val_loss: 1.9612e-04
Epoch 212/350
31/31  0s 4ms/step - loss: 2.1623e-04 - val_loss: 1.9563e-04
Epoch 213/350
31/31  0s 5ms/step - loss: 2.1122e-04 - val_loss: 1.9611e-04
Epoch 214/350
31/31  0s 4ms/step - loss: 2.0964e-04 - val_loss: 1.9558e-04
Epoch 215/350
31/31  0s 4ms/step - loss: 2.0865e-04 - val_loss: 1.9563e-04
Epoch 216/350
31/31  0s 3ms/step - loss: 1.9419e-04 - val_loss: 1.9535e-04
Epoch 217/350
31/31  0s 4ms/step - loss: 2.1475e-04 - val_loss: 1.9530e-04
Epoch 218/350
31/31  0s 4ms/step - loss: 2.1312e-04 - val_loss: 1.9541e-04
Epoch 219/350
31/31  0s 4ms/step - loss: 2.1069e-04 - val_loss: 1.9475e-04
Epoch 220/350
31/31  0s 3ms/step - loss: 2.1244e-04 - val_loss: 1.9465e-04
Epoch 221/350
31/31  0s 4ms/step - loss: 2.0752e-04 - val_loss: 1.9488e-04
Epoch 222/350
31/31  0s 4ms/step - loss: 2.2567e-04 - val_loss: 1.9507e-04
Epoch 223/350
31/31  0s 3ms/step - loss: 2.1963e-04 - val_loss: 1.9451e-04
Epoch 224/350
31/31  0s 4ms/step - loss: 2.0757e-04 - val_loss: 1.9438e-04
Epoch 225/350
31/31  0s 3ms/step - loss: 2.0296e-04 - val_loss: 1.9452e-04
Epoch 226/350
31/31  0s 4ms/step - loss: 2.0718e-04 - val_loss: 1.9388e-04
Epoch 227/350
31/31  0s 3ms/step - loss: 2.2052e-04 - val_loss: 1.9403e-04
Epoch 228/350
31/31  0s 4ms/step - loss: 2.1412e-04 - val_loss: 1.9360e-04
Epoch 229/350
31/31  0s 4ms/step - loss: 2.0990e-04 - val_loss: 1.9378e-04
Epoch 230/350
31/31  0s 4ms/step - loss: 1.9940e-04 - val_loss: 1.9390e-04
Epoch 231/350
31/31  0s 4ms/step - loss: 2.0434e-04 - val_loss: 1.9331e-04
Epoch 232/350
31/31  0s 3ms/step - loss: 2.2831e-04 - val_loss: 1.9321e-04
Epoch 233/350
31/31  0s 4ms/step - loss: 2.0902e-04 - val_loss: 1.9321e-04
Epoch 234/350
31/31  0s 4ms/step - loss: 2.0122e-04 - val_loss: 1.9331e-04
Epoch 235/350
31/31  0s 4ms/step - loss: 2.1467e-04 - val_loss: 1.9304e-04
Epoch 236/350
31/31  0s 4ms/step - loss: 2.0322e-04 - val_loss: 1.9314e-04
Epoch 237/350
31/31  0s 4ms/step - loss: 1.9376e-04 - val_loss: 1.9283e-04
Epoch 238/350
31/31  0s 4ms/step - loss: 2.0861e-04 - val_loss: 1.9266e-04
Epoch 239/350
31/31  0s 4ms/step - loss: 1.8477e-04 - val_loss: 1.9247e-04
Epoch 240/350
31/31  0s 4ms/step - loss: 2.1122e-04 - val_loss: 1.9269e-04
```

```
Epoch 241/350
31/31  0s 4ms/step - loss: 2.1183e-04 - val_loss: 1.9264e-04
Epoch 242/350
31/31  0s 4ms/step - loss: 2.0603e-04 - val_loss: 1.9244e-04
Epoch 243/350
31/31  0s 4ms/step - loss: 2.0928e-04 - val_loss: 1.9231e-04
Epoch 244/350
31/31  0s 4ms/step - loss: 2.2061e-04 - val_loss: 1.9208e-04
Epoch 245/350
31/31  0s 3ms/step - loss: 2.1300e-04 - val_loss: 1.9226e-04
Epoch 246/350
31/31  0s 4ms/step - loss: 2.0945e-04 - val_loss: 1.9201e-04
Epoch 247/350
31/31  0s 4ms/step - loss: 2.0921e-04 - val_loss: 1.9215e-04
Epoch 248/350
31/31  0s 4ms/step - loss: 2.0047e-04 - val_loss: 1.9198e-04
Epoch 249/350
31/31  0s 4ms/step - loss: 2.0061e-04 - val_loss: 1.9182e-04
Epoch 250/350
31/31  0s 4ms/step - loss: 1.9471e-04 - val_loss: 1.9159e-04
Epoch 251/350
31/31  0s 4ms/step - loss: 2.2025e-04 - val_loss: 1.9179e-04
Epoch 252/350
31/31  0s 4ms/step - loss: 2.1785e-04 - val_loss: 1.9166e-04
Epoch 253/350
31/31  0s 4ms/step - loss: 1.9310e-04 - val_loss: 1.9155e-04
Epoch 254/350
31/31  0s 4ms/step - loss: 1.9552e-04 - val_loss: 1.9153e-04
Epoch 255/350
31/31  0s 4ms/step - loss: 2.0099e-04 - val_loss: 1.9164e-04
Epoch 256/350
31/31  0s 3ms/step - loss: 1.9817e-04 - val_loss: 1.9139e-04
Epoch 257/350
31/31  0s 4ms/step - loss: 2.2082e-04 - val_loss: 1.9148e-04
Epoch 258/350
31/31  0s 3ms/step - loss: 2.1275e-04 - val_loss: 1.9137e-04
Epoch 259/350
31/31  0s 4ms/step - loss: 2.1050e-04 - val_loss: 1.9125e-04
Epoch 260/350
31/31  0s 3ms/step - loss: 2.0023e-04 - val_loss: 1.9105e-04
Epoch 261/350
31/31  0s 5ms/step - loss: 2.0978e-04 - val_loss: 1.9092e-04
Epoch 262/350
31/31  0s 4ms/step - loss: 2.1119e-04 - val_loss: 1.9122e-04
Epoch 263/350
31/31  0s 4ms/step - loss: 1.9421e-04 - val_loss: 1.9110e-04
Epoch 264/350
31/31  0s 4ms/step - loss: 2.2247e-04 - val_loss: 1.9086e-04
Epoch 265/350
31/31  0s 4ms/step - loss: 2.0417e-04 - val_loss: 1.9094e-04
Epoch 266/350
31/31  0s 4ms/step - loss: 2.1107e-04 - val_loss: 1.9087e-04
Epoch 267/350
31/31  0s 4ms/step - loss: 2.0984e-04 - val_loss: 1.9075e-04
Epoch 268/350
31/31  0s 4ms/step - loss: 2.0120e-04 - val_loss: 1.9068e-04
Epoch 269/350
31/31  0s 4ms/step - loss: 2.0953e-04 - val_loss: 1.9071e-04
Epoch 270/350
31/31  0s 4ms/step - loss: 2.1178e-04 - val_loss: 1.9047e-04
```

```
Epoch 271/350
31/31  0s 4ms/step - loss: 1.9452e-04 - val_loss: 1.9066e-04
Epoch 272/350
31/31  0s 4ms/step - loss: 2.1369e-04 - val_loss: 1.9071e-04
Epoch 273/350
31/31  0s 4ms/step - loss: 2.0784e-04 - val_loss: 1.9045e-04
Epoch 274/350
31/31  0s 4ms/step - loss: 2.0072e-04 - val_loss: 1.9014e-04
Epoch 275/350
31/31  0s 4ms/step - loss: 2.0140e-04 - val_loss: 1.9037e-04
Epoch 276/350
31/31  0s 4ms/step - loss: 2.0593e-04 - val_loss: 1.9065e-04
Epoch 277/350
31/31  0s 4ms/step - loss: 2.0505e-04 - val_loss: 1.9022e-04
Epoch 278/350
31/31  0s 4ms/step - loss: 1.9466e-04 - val_loss: 1.9002e-04
Epoch 279/350
31/31  0s 4ms/step - loss: 2.0033e-04 - val_loss: 1.9028e-04
Epoch 280/350
31/31  0s 4ms/step - loss: 1.8371e-04 - val_loss: 1.9030e-04
Epoch 281/350
31/31  0s 3ms/step - loss: 2.1231e-04 - val_loss: 1.8999e-04
Epoch 282/350
31/31  0s 4ms/step - loss: 1.9531e-04 - val_loss: 1.9046e-04
Epoch 283/350
31/31  0s 4ms/step - loss: 2.1411e-04 - val_loss: 1.9029e-04
Epoch 284/350
31/31  0s 4ms/step - loss: 2.1142e-04 - val_loss: 1.9007e-04
Epoch 285/350
31/31  0s 3ms/step - loss: 2.0217e-04 - val_loss: 1.9016e-04
Epoch 286/350
31/31  0s 4ms/step - loss: 2.1213e-04 - val_loss: 1.8995e-04
Epoch 287/350
31/31  0s 4ms/step - loss: 1.8802e-04 - val_loss: 1.8999e-04
Epoch 288/350
31/31  0s 4ms/step - loss: 2.0642e-04 - val_loss: 1.9021e-04
Epoch 289/350
31/31  0s 4ms/step - loss: 2.0092e-04 - val_loss: 1.9008e-04
Epoch 290/350
31/31  0s 4ms/step - loss: 1.9661e-04 - val_loss: 1.8975e-04
Epoch 291/350
31/31  0s 4ms/step - loss: 1.9533e-04 - val_loss: 1.8989e-04
Epoch 292/350
31/31  0s 4ms/step - loss: 2.1424e-04 - val_loss: 1.8968e-04
Epoch 293/350
31/31  0s 4ms/step - loss: 2.0287e-04 - val_loss: 1.8986e-04
Epoch 294/350
31/31  0s 4ms/step - loss: 2.0000e-04 - val_loss: 1.9010e-04
Epoch 295/350
31/31  0s 4ms/step - loss: 2.0037e-04 - val_loss: 1.8980e-04
Epoch 296/350
31/31  0s 4ms/step - loss: 2.0847e-04 - val_loss: 1.8980e-04
Epoch 297/350
31/31  0s 4ms/step - loss: 1.9677e-04 - val_loss: 1.8966e-04
Epoch 298/350
31/31  0s 4ms/step - loss: 1.9978e-04 - val_loss: 1.8975e-04
Epoch 299/350
31/31  0s 4ms/step - loss: 2.0915e-04 - val_loss: 1.8989e-04
Epoch 300/350
31/31  0s 4ms/step - loss: 1.9817e-04 - val_loss: 1.8949e-04
```

```
Epoch 301/350
31/31  0s 4ms/step - loss: 2.0183e-04 - val_loss: 1.8987e-04
Epoch 302/350
31/31  0s 4ms/step - loss: 2.1049e-04 - val_loss: 1.8973e-04
Epoch 303/350
31/31  0s 4ms/step - loss: 2.0544e-04 - val_loss: 1.8961e-04
Epoch 304/350
31/31  0s 4ms/step - loss: 1.9764e-04 - val_loss: 1.8942e-04
Epoch 305/350
31/31  0s 3ms/step - loss: 1.9379e-04 - val_loss: 1.8957e-04
Epoch 306/350
31/31  0s 4ms/step - loss: 2.0660e-04 - val_loss: 1.8942e-04
Epoch 307/350
31/31  0s 4ms/step - loss: 2.0000e-04 - val_loss: 1.8958e-04
Epoch 308/350
31/31  0s 3ms/step - loss: 1.9812e-04 - val_loss: 1.8942e-04
Epoch 309/350
31/31  0s 4ms/step - loss: 1.9726e-04 - val_loss: 1.8940e-04
Epoch 310/350
31/31  0s 6ms/step - loss: 2.2123e-04 - val_loss: 1.8947e-04
Epoch 311/350
31/31  0s 5ms/step - loss: 2.1080e-04 - val_loss: 1.8898e-04
Epoch 312/350
31/31  0s 4ms/step - loss: 2.1011e-04 - val_loss: 1.8994e-04
Epoch 313/350
31/31  0s 4ms/step - loss: 2.1423e-04 - val_loss: 1.8910e-04
Epoch 314/350
31/31  0s 4ms/step - loss: 1.9890e-04 - val_loss: 1.8919e-04
Epoch 315/350
31/31  0s 5ms/step - loss: 1.9970e-04 - val_loss: 1.8936e-04
Epoch 316/350
31/31  0s 4ms/step - loss: 2.1570e-04 - val_loss: 1.8908e-04
Epoch 317/350
31/31  0s 4ms/step - loss: 2.1581e-04 - val_loss: 1.8916e-04
Epoch 318/350
31/31  0s 3ms/step - loss: 1.9793e-04 - val_loss: 1.8928e-04
Epoch 319/350
31/31  0s 4ms/step - loss: 1.9354e-04 - val_loss: 1.8905e-04
Epoch 320/350
31/31  0s 4ms/step - loss: 2.0648e-04 - val_loss: 1.8890e-04
Epoch 321/350
31/31  0s 4ms/step - loss: 2.0039e-04 - val_loss: 1.8914e-04
Epoch 322/350
31/31  0s 4ms/step - loss: 2.0062e-04 - val_loss: 1.8893e-04
Epoch 323/350
31/31  0s 4ms/step - loss: 2.0052e-04 - val_loss: 1.8900e-04
Epoch 324/350
31/31  0s 4ms/step - loss: 2.0428e-04 - val_loss: 1.8899e-04
Epoch 325/350
31/31  0s 4ms/step - loss: 1.8988e-04 - val_loss: 1.8915e-04
Epoch 326/350
31/31  0s 4ms/step - loss: 2.1172e-04 - val_loss: 1.8897e-04
Epoch 327/350
31/31  0s 4ms/step - loss: 1.9617e-04 - val_loss: 1.8871e-04
Epoch 328/350
31/31  0s 4ms/step - loss: 2.0471e-04 - val_loss: 1.8886e-04
Epoch 329/350
31/31  0s 5ms/step - loss: 2.0391e-04 - val_loss: 1.8920e-04
Epoch 330/350
31/31  0s 4ms/step - loss: 2.1750e-04 - val_loss: 1.8889e-04
```

```

Epoch 331/350
31/31  0s 4ms/step - loss: 1.8783e-04 - val_loss: 1.8890e-04
Epoch 332/350
31/31  0s 4ms/step - loss: 1.9765e-04 - val_loss: 1.8906e-04
Epoch 333/350
31/31  0s 3ms/step - loss: 2.0759e-04 - val_loss: 1.8882e-04
Epoch 334/350
31/31  0s 4ms/step - loss: 1.9244e-04 - val_loss: 1.8888e-04
Epoch 335/350
31/31  0s 4ms/step - loss: 2.1368e-04 - val_loss: 1.8897e-04
Epoch 336/350
31/31  0s 3ms/step - loss: 2.1242e-04 - val_loss: 1.8894e-04
Epoch 337/350
31/31  0s 4ms/step - loss: 2.0768e-04 - val_loss: 1.8857e-04
Epoch 338/350
31/31  0s 4ms/step - loss: 2.1237e-04 - val_loss: 1.8893e-04
Epoch 339/350
31/31  0s 4ms/step - loss: 2.0179e-04 - val_loss: 1.8875e-04
Epoch 340/350
31/31  0s 4ms/step - loss: 1.9195e-04 - val_loss: 1.8893e-04
Epoch 341/350
31/31  0s 4ms/step - loss: 1.8425e-04 - val_loss: 1.8900e-04
Epoch 342/350
31/31  0s 4ms/step - loss: 1.9478e-04 - val_loss: 1.8862e-04
Epoch 343/350
31/31  0s 4ms/step - loss: 1.9228e-04 - val_loss: 1.8914e-04
Epoch 344/350
31/31  0s 4ms/step - loss: 2.0520e-04 - val_loss: 1.8873e-04
Epoch 345/350
31/31  0s 4ms/step - loss: 2.1454e-04 - val_loss: 1.8882e-04
Epoch 346/350
31/31  0s 4ms/step - loss: 2.1062e-04 - val_loss: 1.8889e-04
Epoch 347/350
31/31  0s 4ms/step - loss: 1.8721e-04 - val_loss: 1.8845e-04
Epoch 348/350
31/31  0s 4ms/step - loss: 2.0844e-04 - val_loss: 1.8903e-04
Epoch 349/350
31/31  0s 4ms/step - loss: 2.0617e-04 - val_loss: 1.8870e-04
Epoch 350/350
31/31  0s 4ms/step - loss: 1.9899e-04 - val_loss: 1.8859e-04

```

Out[19]: <keras.src.callbacks.history.History at 0x7968c842fe50>

```

In [20]: Y_pred = model_nonlinear.predict(Y_train, verbose=0)
error_non = np.sum((Y_train - Y_pred) ** 2) / np.sum((Y_train - np.mean(Y_train)
print("Reconstruction error in training sample", error_non)
Y_pred = model_nonlinear.predict(Y_test, verbose=0)
error_non_oos = np.sum((Y_test - Y_pred) ** 2) / np.sum((Y_test - np.mean(Y_test)
print("Reconstruction error in test sample", error_non_oos)

```

```

Reconstruction error in training sample 0.3094527914957498
Reconstruction error in test sample 0.4788719014902

```

```

In [21]: factors = model_nonlinear.encoder(Y_train).numpy()
factors_test = model_nonlinear.encoder(Y_test).numpy()
print(np.cov(factors, rowvar=False))

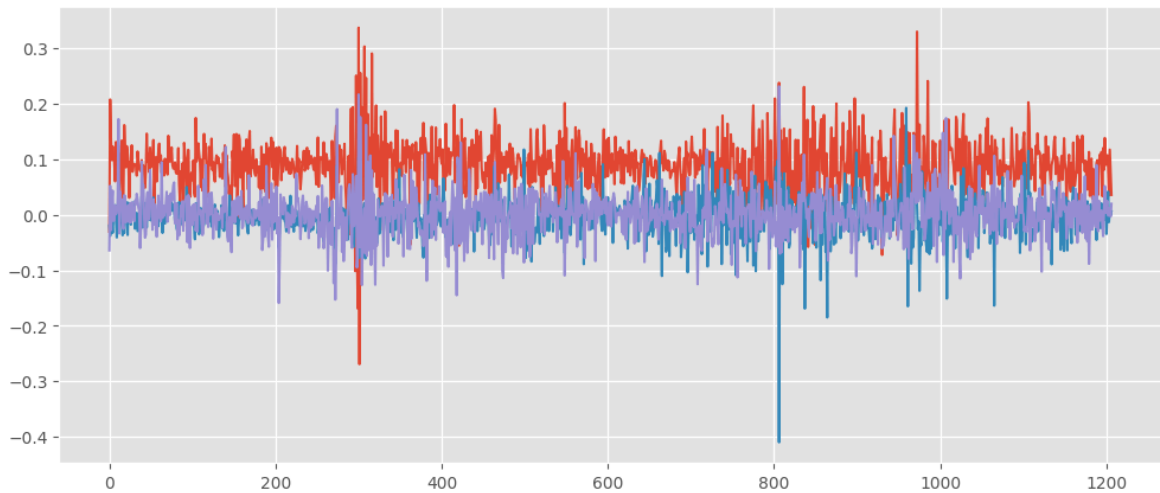
```

```

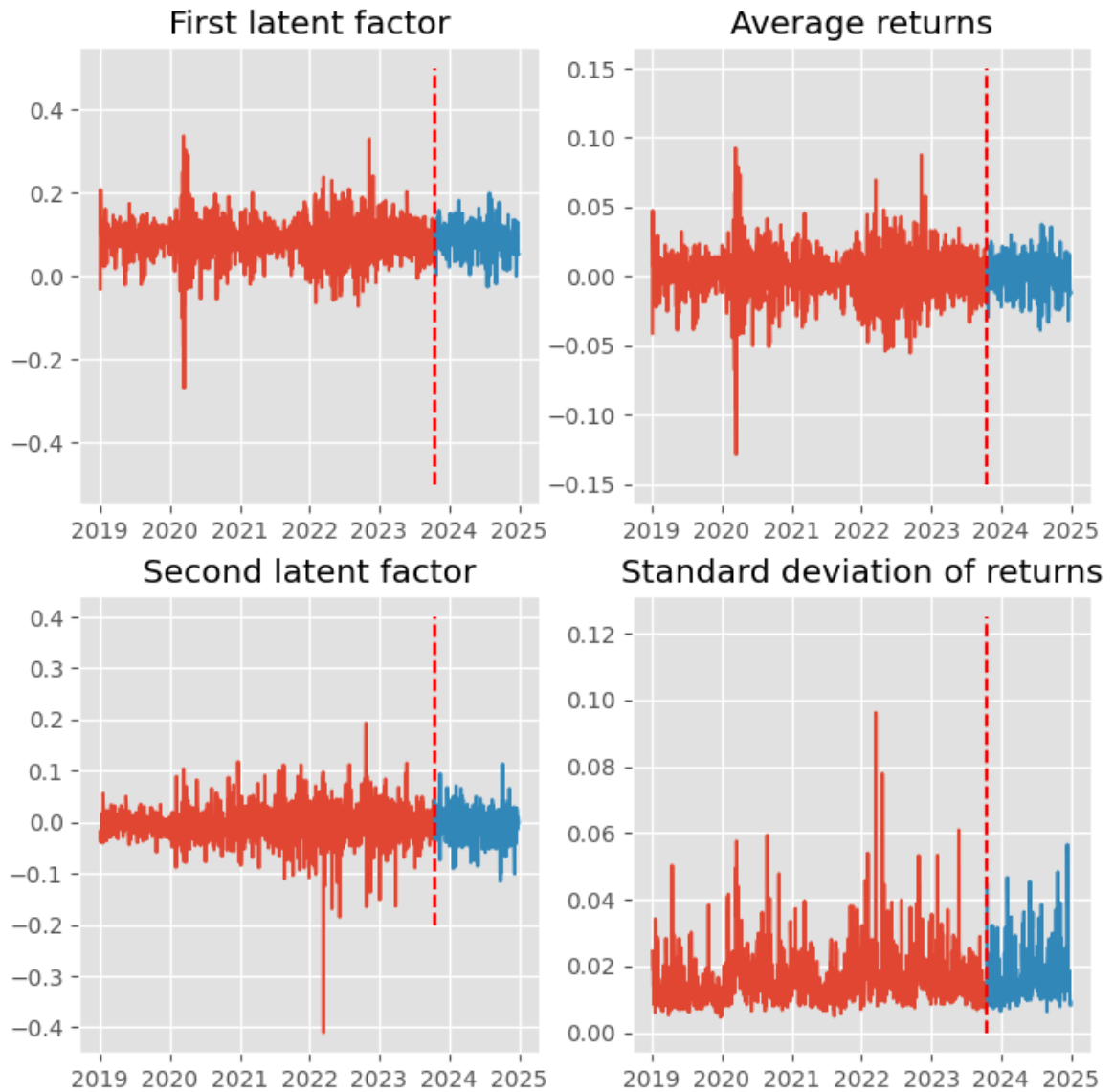
[[ 2.41902423e-03  1.45583215e-04  5.84779443e-04]
 [ 1.45583215e-04  1.35614948e-03 -5.56242140e-06]
 [ 5.84779443e-04 -5.56242140e-06  1.74229230e-03]]

```

```
In [22]: # Plot the evolution of latent factors in the training period
plt.figure(figsize=(12, 5))
plt.plot(factors)
plt.show()
```

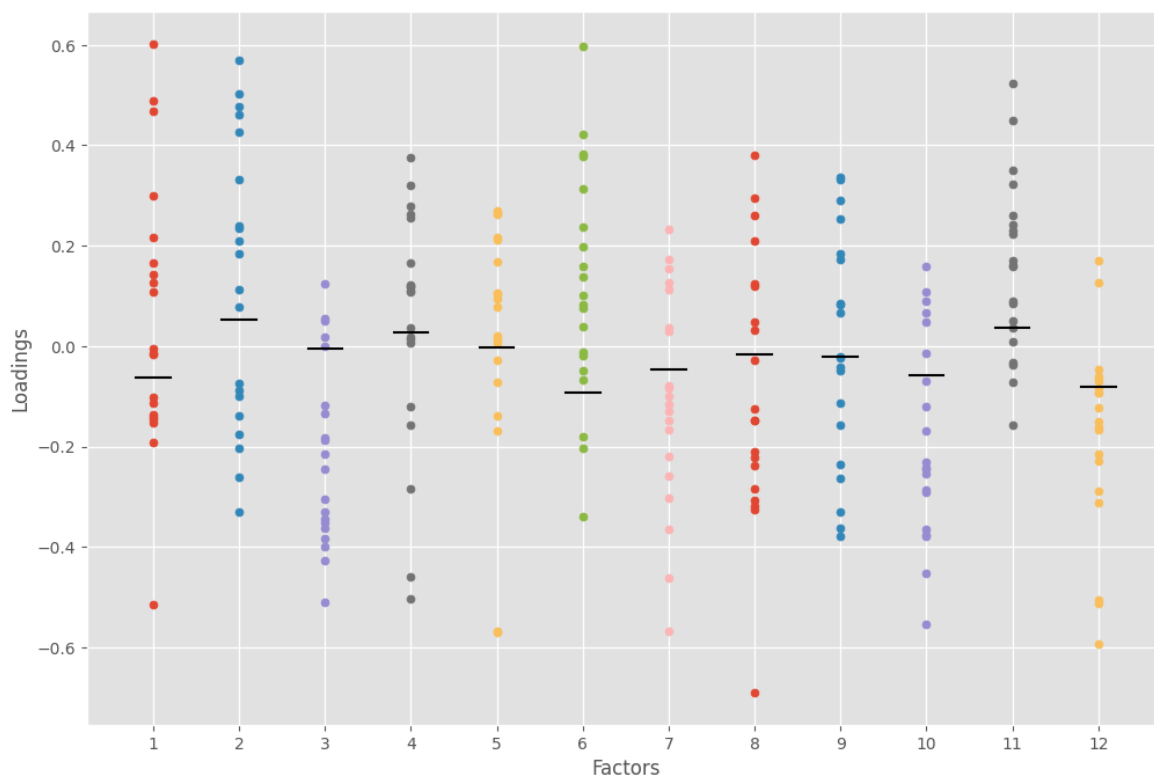


```
In [23]: fig, axs = plt.subplots(2, 2, figsize=(8, 8))
# First factor
axs[0, 0].plot(df.index[: train_size + 1], factors[:, 0])
axs[0, 0].plot(df.index[train_size + 1 :], factors_test[:, 0])
axs[0, 0].plot([df.index[train_size + 1], df.index[train_size + 1]], [-0.5, 0.5])
axs[0, 0].title.set_text("First latent factor")
# Average of returns
axs[0, 1].plot(df.index[: train_size + 1], np.mean(Y_train, axis=1))
axs[0, 1].plot(df.index[train_size + 1 :], np.mean(Y_test, axis=1))
axs[0, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [-0.15, 0.15], "r--"
)
axs[0, 1].title.set_text("Average returns")
# Second factor
axs[1, 0].plot(df.index[: train_size + 1], factors[:, 1])
axs[1, 0].plot(df.index[train_size + 1 :], factors_test[:, 1])
axs[1, 0].plot([df.index[train_size + 1], df.index[train_size + 1]], [-0.2, 0.4])
axs[1, 0].title.set_text("Second latent factor")
# Standard deviation of returns
axs[1, 1].plot(df.index[: train_size + 1], np.std(Y_train, axis=1))
axs[1, 1].plot(df.index[train_size + 1 :], np.std(Y_test, axis=1))
axs[1, 1].plot(
    [df.index[train_size + 1], df.index[train_size + 1]], [0.0, 0.125], "r--"
)
axs[1, 1].title.set_text("Standard deviation of returns")
```



```
In [24]: # Extract Loadings generated by the autoencoder in the output layer
w_decoder = model_nonlinear.decoder.layers[1].get_weights()[0]
data = w_decoder
labels = list(range(1, 13))
width = 0.4
fig, ax = plt.subplots(figsize=(12, 8))
for i, l in enumerate(labels):
    x = np.ones(data.shape[1]) * i
    ax.scatter(x, data[i, :], s=25)
    mean = data[:, i].mean()
    ax.plot([i - width / 2.0, i + width / 2.0], [mean, mean], color="k")
    ax.set_xlabel("Factors")
    ax.set_ylabel("Loadings")
ax.set_xticks(range(len(labels)))
ax.set_xticklabels(labels)

plt.show()
```



5. CONCLUSION

In this notebook, we have learned to design and train autoencoders with TensorFlow as a tool to extract the main latent factors that drive the variation of financial time series.

In the next lesson, we will explore tools to refine autoencoders and describe other types of architectures.

See you there!

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.